ADL Lite: An Event-First Operational Ontology for Concept Lifecycle Governance

Anonymous Authors
Affiliation withheld for peer review

Abstract

This paper presents ADL Lite, an event-first operational ontology for concept lifecycle governance. Drawing on Wittgenstein's thesis that "the world is the totality of facts, not of things," ADL Lite treats events—not objects—as the fundamental ontological primitive. Concepts are modeled as append-only, cryptographically linked EventChains, with all lifecycle state (status, confidence, validators) derived exclusively from event history through deterministic functions. This design eliminates stored mutable state and the consistency bugs it engenders.

The paper's primary contribution is an ontological analysis that situates ADL Lite within the landscape of foundational ontologies. We demonstrate that ADL Lite's Event corresponds to the BFO category of *occurrent* and the DOLCE category of *perdurant*; that its Concept is best understood as a BFO *generically dependent continuant* or a DOLCE *non-physical object*; and that its Relation blocks instantiate UFO *relators*. Identity conditions for events, concepts, and chains are formalized, and three forms of ontological dependence (historical, generic, mutual) are analyzed. Alignment choices and intentional deviations from BFO, DOLCE, and UFO are discussed and justified.

A complete formal semantics is provided: an event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$, a well-formedness predicate $\text{WF}(C)$, status derivation $\delta(C)$, confidence aggregation $\gamma(C)$, and fork/confluence semantics. Six properties are proved: determinism (Theorem 1), confluence under fork (Theorem 2), transition monotonicity (Theorem 3), confidence boundedness (Theorem 4), confidence monotonicity under independent validation (Theorem 5), and status–confidence consistency (Theorem 6). The expressive power of the derivation functions is analyzed in relation to Event Calculus and Description Logic.

Empirical validation on 201 EventChains derived from the IBM Anti-Money Laundering dataset (9,300 events) confirms zero integrity failures and linear scalability. The ontology is implemented as a pip-installable Python toolkit operating over Markdown files in standard Git repositories, with no external triple store or OWL reasoner required.

Keywords: ontology engineering; event sourcing; conceptual modeling; applied ontology; multi-agent systems; knowledge graphs; provenance; BFO; DOLCE; UFO

1 Introduction

1.1 Background: From Object-First to Event-First Ontology Engineering

Ontology engineering has traditionally adopted an *object-first* stance. In this paradigm, the ontologist begins by identifying the kinds of entities that exist in a domain (Object Types), their attributes (Property Types), the relationships among them (Link Types), and only subsequently models the operations that can be performed upon these structures (Action Types) W3C OWL Working Group [2012], Palantir Technologies [2024]. This approach underpins the Web Ontology Language (OWL) W3C OWL Working Group [2012], enterprise knowledge graphs, and industrial platforms such as

Palantir Foundry Data Engine Palantir Technologies [2024]. It reflects a metaphysical commitment to the primacy of *continuants*—entities that exist in full at every moment of their existence and persist through changes in their attributes.

The object-first paradigm has proven effective in domains where the entity types are stable and well-understood: biomedicine (Gene Ontology), cultural heritage (CIDOC CRM), and enterprise data management (Palantir FDE). However, it encounters fundamental difficulties in domains characterized by high conceptual velocity, where new entity types emerge frequently and existing types evolve rapidly. Multi-agent systems powered by large language models (LLMs) exemplify such domains: autonomous agents generate hypotheses, propose concepts, validate one another's contributions, and deprecate outdated ideas at a rate that exceeds human curation capacity LangChain, Inc. [2023], OpenAI [2024]. In this setting, the object-first paradigm creates a bottleneck: every new concept type must be manually defined before instances can be created, and every status transition must be explicitly modeled as a permissible operation on an object state.

A second limitation of the object-first paradigm concerns the treatment of *state*. In conventional ontology platforms, a concept's status (e.g., `proposed`, `validated`, `deprecated`) is stored as a mutable field. This field can diverge from the concept's actual history when updates occur outside the governed action pathway—through race conditions between concurrent agents, manual overrides, or bugs in external systems. The resulting inconsistency is not merely a software bug; it is an ontological problem: the stored state no longer accurately represents the entity's true ontological status, which is constituted by the history of events that have affected it.

Ontological novelty of the event-first stance. We emphasize that ADL Lite's contribution is not merely adopting "event" as an ontological category—a position already well-established by BFO's *occurrent*, DOLCE's *perdurant*, and UFO-B's *event* **???**. Rather, ADL Lite advances a *paradigm-level reduction*: events are not merely one category among others (alongside continuants, endurants, and qualities), but the *sole primitive* from which all other ontological categories are derived. Concepts, relations, statuses, and confidence values are all projections of event patterns, not independent entities with intrinsic properties. This is analogous to the reduction of chemistry to physics: atoms are not eliminated, but understood as composites of more fundamental entities. In ADL Lite, a "concept" has no ontological standing independent of its EventChain; its identity, status, and confidence are constituted entirely by the events that have occurred in its lifecycle. This reduction distinguishes ADL Lite from foundational ontologies that treat events and continuants as co-equal top-level categories, and it motivates the architectural design choices—append-only chains, hash-linked integrity, and deterministic derivation—that constitute the technical contribution of this work.

1.2 Research Gap: Concept Lifecycle Governance as Ontological Analysis

The specific research gap addressed in this paper can be formulated as follows: *How can an operational ontology achieve lifecycle traceability and multi-agent governance without requiring object-first mutable state, expert curation, or heavyweight tooling?* This gap has both an engineering dimension and an ontological dimension.

The **engineering dimension** concerns deployment friction. Existing operational ontology platforms require specialized expertise, dedicated infrastructure, and manual curation workflows that do not scale to the velocity of agent-generated conceptual output **??**. LLM-based agents cannot participate in ontology construction because they lack programmatic access to the proprietary interfaces through which ontologies are defined.

The **ontological dimension** concerns the status–history relationship. In an object-first ontology, an entity's status is a property of the entity. In an event-first ontology, an entity's status is a derivative of its history. The transition from object-first to event-first is not merely a software design pattern (though it resembles event sourcing Young [2010, 2012]); it is an ontological reorientation that changes the fundamental categories of the ontology. This reorientation has not been systematically analyzed in the applied ontology literature.

This paper addresses both dimensions by presenting ADL Lite, an event-first operational ontology in which concepts are modeled as append-only, cryptographically linked EventChains, and all lifecycle state is derived exclusively from event history. The paper's primary contribution is an **ontological analysis** that situates ADL Lite within the landscape of foundational ontologies (BFO, DOLCE, UFO), formalizes its categories and identity conditions, and proves key properties of its derivation semantics.

1.3 Contributions

The contributions of this paper are organized around three axes: ontological analysis, formal semantics, and empirical validation.

Contribution 1: Ontological analysis of event-first concept governance. We analyze ADL Lite's core categories through the lens of three influential foundational ontologies:

- We show that ADL Lite's `Event` corresponds to the BFO category of *occurrent* and the DOLCE category of *perdurant*.
- We argue that ADL Lite's `Concept` is best understood as a BFO *generically dependent continuant* or a DOLCE *non-physical object*.
- We demonstrate that ADL Lite's L3 relations instantiate UFO *relators* with event-based lifecycles.
- We formalize identity conditions for events, concepts, and chains, and analyze three forms of ontological dependence (historical, generic, mutual).

Contribution 2: Formal derivation semantics with proved properties. We provide a complete formal foundation comprising:

- An event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ partitioned into lifecycle and communication events.
- A well-formedness predicate $\text{WF}(C)$ over EventChains.
- Status derivation $\delta(C)$ and confidence aggregation $\gamma(C)$ as deterministic functions.
- Fork and confluence semantics.
- Six proved properties: determinism (Theorem 1), confluence under fork (Theorem 2), transition monotonicity (Theorem 3), confidence boundedness (Theorem 4), confidence monotonicity under independent validation (Theorem 5), and status–confidence consistency (Theorem 6).
- An analysis of expressive power in relation to Event Calculus and Description Logic.

Contribution 3: Empirical validation of formal semantics and ontological consistency. We evaluate ADL Lite on 201 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset (9,300 events), empirically validating the formal semantics with six proved theorems from Contribution 2. The evaluation confirms:

- Zero integrity failures across all chains, including the longest chain (300 events).
- 100% accuracy on 2,204 exhaustive status derivation test cases.
- Perfect precision and recall (P = R = F1 = 1.0) on 13 precondition enforcement test cases.
- Linear scalability: 31,100 events per second on modest hardware.

These results do not constitute domain-level evaluation of AML effectiveness; they validate that the event-first ontological architecture—including the deterministic derivation semantics, hash-linked integrity guarantees, and six formally proved properties—is computationally feasible, cryptographically sound, and empirically correct.

1.4 Paper Organization

Section 2 surveys related work in foundational ontologies, ontology engineering, and event-centric modeling. Section 3 presents the core ontological analysis: categories, identity conditions, ontological dependence, and alignment with BFO, DOLCE, and UFO. Section 4 describes the ADL Lite architecture, formal semantics, and comparison with Event Calculus and Description Logic. Section 5 reports empirical validation. Section 6 discusses implications and limitations. Section 7 concludes and identifies future work.

2 Related Work

2.1 Upper Ontologies and Foundational Ontologies

Foundational ontologies provide top-level categories that constrain and guide domain ontology development. Three influential frameworks—Basic Formal Ontology (BFO), the Descriptive Ontology for Linguistic and Cognitive Engineering (DOLCE), and the Unified Foundational Ontology (UFO)—offer distinct but complementary perspectives on the categories of entity, the nature of events, and the relationship between objects and processes. Understanding ADL Lite's position with respect to these frameworks requires examining their treatment of *occurrents* (temporal entities), *continuants* (enduring entities), and the ontological status of information objects.

2.1.1 Basic Formal Ontology (BFO)

BFO is a top-level ontology developed primarily for the biomedical and life-science domains **?**. Its core distinction is between *continuants* (entities that exist in full at every moment of their existence) and *occurrents* (entities that unfold in time and have temporal parts). Continuants include independent continuants (material objects), dependent continuants (qualities, roles, dispositions), and generically dependent continuants (information content entities). Occurrents include processes (events with temporal extent).

BFO's treatment of events is particularly relevant to ADL Lite. In BFO 2.0, a *process* is an occurrent that has a temporal extent and at least one participant that is a continuant **?**. Processes are not changes of state; they are entities in their own right. A validation process, for instance, is not merely the transition of a concept from `provisional` to `validated`; it is an occurrent with its own identity, participants (the validating agent, the concept being validated), and temporal boundaries.

ADL Lite's `Event` corresponds closely to BFO's *process*. Both are occurrents with temporal extent and participants. However, ADL Lite does not adopt BFO's full continuant/occurrent

hierarchy; instead, it collapses the distinction into a single event model in which all state changes are represented as events. This simplification is motivated by the target domain (multi-agent concept governance), where the primary concern is recording what happened, not classifying entities into a complex hierarchy of types.

A second BFO concept relevant to ADL Lite is the *generically dependent continuant* (GDC). A GDC depends on some continuant for its existence, but not on any specific continuant **?**. A poem, for instance, is a GDC because it depends on some physical copy (a book, a memory) but not on any specific copy. ADL Lite's "concept" is best understood as a GDC: it depends on its EventChain for existence, but not on any specific event within the chain.

2.1.2 DOLCE

DOLCE is a foundational ontology designed with a strong cognitive-linguistic orientation **?**. Its top-level categories include *particulars* (entities that exist in space and time) and *universals* (entities that can be instantiated). Among particulars, DOLCE distinguishes *perdurants* (entities that are only partially present at any time) from *endurants* (entities that are wholly present at any time).

DOLCE's category of *perdurant* corresponds to BFO's *occurrent* and ADL Lite's `Event`. However, DOLCE makes finer-grained distinctions among perdurants: *events* (changes of state), *processes* (activities with no definite endpoint), *states* (stable conditions), and *accomplishments* (activities with a definite endpoint) **?**. ADL Lite's event types do not explicitly adopt this taxonomy, but they can be mapped to it: `REGISTER` and `VALIDATE` are events (changes of state), while `ANNOUNCE` and `PUBLISH` are accomplishments (activities with a definite endpoint).

DOLCE's category of *non-physical object* (NPO) is also relevant. NPOs include social objects (e.g., organizations, contracts), information objects (e.g., documents, databases), and cognitive objects (e.g., thoughts, beliefs) **?**. ADL Lite concepts are information objects that carry social weight (their status reflects community acceptance). In DOLCE terms, they are NPOs that depend on a community of agents for their interpretation and validation.

A key DOLCE principle is *descriptive adequacy*: the ontology should capture the categories that humans naturally use to conceptualize the world **?**. ADL Lite's document model (L1–L4) is designed with this principle in mind: the layers correspond to human cognitive distinctions between identity (L1), narrative (L2), assertions (L3), and actions (L4). This alignment is intentional: ADL Lite targets human-LLM collaborative authoring, so its categories must be cognitively accessible.

2.1.3 Unified Foundational Ontology (UFO)

UFO is a foundational ontology developed specifically for conceptual modeling and software engineering **?**. It combines insights from BFO, DOLCE, and cognitive linguistics into a framework designed for building domain ontologies and conceptual models. UFO's distinctive contribution is its treatment of *types*, *roles*, *phases*, and *mixins*—categories that are essential for modeling dynamic domains but are not explicitly represented in BFO or DOLCE.

UFO distinguishes three ontological levels: *individuals* (particular entities), *types* (categories that classify individuals), and *concepts* (mental representations of types) **?**. ADL Lite operates primarily at the type level: `concept_id` identifies a type (e.g., "Capital Trap"), and the EventChain records the history of that type. However, ADL Lite also has elements of the individual level: each event is an individual occurrent, and each concept instance (e.g., a specific application of the Capital Trap pattern) is an individual.

UFO's treatment of *events* and *actions* is particularly relevant. In UFO, an event is a change in the world, while an action is an intentional event performed by an agent to achieve a goal **?**. ADL

Lite's event type system captures this distinction: `REGISTER`, `VALIDATE`, `DEPRECATE` are actions (intentional, performed by agents), while `RELATE`, `EVIDENCE`, `SEAL` are events (assertions that change relational structure). The precondition system in ADL Lite reflects the *plan* aspect of actions: an action is not merely a change; it is a change that is governed by norms.

UFO's category of *relator* is also relevant to ADL Lite's L3 relation blocks. A relator is an entity that mediates the relationship between two or more entities **?**. In ADL Lite, a relation between two concepts is established by a `RELATE` event and can be revoked by a future event. This event-based lifecycle aligns with UFO's view that relators are brought into existence and destroyed by events.

2.1.4 Synthesis: ADL Lite's Position in the Foundational Ontology Landscape

Table 1 summarizes the comparison across the three foundational ontologies.

Table 1: Comparison of foundational ontology perspectives on ADL Lite categories.

Feature	**BFO**	**DOLCE**	**UFO**
Event category	Process (occurrent)	Perdurant (event)	Event / Action
Concept category	Generically dependent continuant	Non-physical object (information)	Conceptual entity / Type
Relation category	Relational quality	Quality (relational)	Relator
Identity of concept	concept_id (rigid designator)	Social object identity	Type identity
Temporal reasoning	Process parthood	Temporal part	Event sequence
Agent involvement	Process has participant	Agentive physical object	Agent performs action

ADL Lite does not fully align with any single foundational ontology. Instead, it adopts a *pragmatic pluralism*: it uses BFO's continuant/occurrent distinction as a conceptual guide, DOLCE's descriptive adequacy as a design principle, and UFO's event/action distinction as a modeling framework. This pluralism is deliberate: the target domain (multi-agent concept governance) requires a lightweight ontology that can be authored by non-experts in Markdown, not a heavyweight foundational ontology that requires expert curation in OWL.

The cost of this pluralism is that ADL Lite cannot directly import BFO, DOLCE, or UFO modules without translation. The benefit is that the ontology is optimized for its intended use case: lightweight, human-LLM collaborative, event-first concept governance. Section 3 provides the detailed ontological analysis that justifies these alignment choices.

2.2 Ontology Engineering and Knowledge Graph Construction

The Semantic Web vision articulated by Berners-Lee, Hendler, and Lassila **?** established the foundational goal of machine-readable, interlinked knowledge representations. This vision was operationalized through OWL 2, whose formal semantics rooted in description logics provide expressiveness ranging from the lightweight OWL 2 EL profile to the fully decidable OWL 2 DL W3C OWL Working Group [2012]. Standard toolchains including Protégé, Apache Jena, and Virtuoso Universal Server enable ontology authoring, persistence, and SPARQL query evaluation at considerable scale. Reasoners such as HermiT and Pellet support automated classification and

consistency checking, albeit with computational costs that scale poorly for large or rapidly evolving knowledge bases **?**.

However, the deployment cost of full OWL-based pipelines remains prohibitive for many operational contexts. Triple stores require dedicated infrastructure; OWL reasoners impose batch-oriented computation ill-suited to streaming workloads; and the expertise required for ontology engineering creates a bottleneck distinct from application development **?**. These factors confine heavyweight Semantic Web technologies to domains where upfront investment in formal knowledge representation is justified by long-term reasoning requirements.

In response, the community has developed lightweight approaches that trade full logical expressiveness for accessibility. Schema.org provides a pragmatic, property-centric vocabulary adopted on over 40% of web pages **?**. JSON-LD extends conventional JSON with namespace support, allowing RDF-compatible serialization without requiring full Linked Data processing Sporny et al. [2020]. SHACL provides data validation without inference, enabling closed-world constraint checking over RDF graphs W3C [2017]. These technologies lower the barrier to structured data publishing but remain tethered to the RDF data model, excluding the vast corpus of domain knowledge encoded in Markdown documentation and plain-text notes.

OBO Foundry and Ontology Change Management. The Open Biological and Biomedical Ontologies (OBO) Foundry provides principles for ontology development, including versioning policy, identifier permanence, and change requests tracked through GitHub issues. Systems such as OORT (Ontology Release Tool) and OntoFox manage large-scale ontology imports and ensure that terms retain persistent identifiers across releases. ADL Lite shares OBO Foundry's commitment to identifier permanence and auditable change but differs in mechanism: OBO uses centralized release pipelines with curator-reviewed pull requests, while ADL Lite uses decentralized append-only EventChains with cryptographic hash linkage. The EventChain approach enables automatic verification of provenance without relying on a central curator to gate changes. However, ADL Lite currently lacks OBO Foundry's mature import and reuse infrastructure—a gap scoped to future work.

RO-Crate and FAIR Digital Objects. RO-Crate provides a lightweight approach to packaging research outputs with standardized metadata, using JSON-LD to describe datasets, workflows, and their provenance. FAIR Digital Objects extend this concept to distributed persistent identifiers with rich metadata. Both frameworks share ADL Lite's goal of making provenance machine-accessible, but differ in scope: RO-Crate packages research outputs at rest, while ADL Lite governs knowledge artifacts in flight—capturing the full lifecycle from proposal to validation to deprecation as an auditable event sequence. RO-Crate's metadata profiles could be generated from ADL Lite EventChains, providing a bridge from active governance to archival publishing.

The past decade has witnessed significant growth in Markdown-based knowledge management tools. Foam, Obsidian, and Dendron extend plain-text Markdown with wiki-style links, backlinks, and graph visualization, enabling knowledge workers to construct personal knowledge graphs without leaving their editing environment **?**. Dendron introduces hierarchical note schemas with refactoring capabilities that preserve link integrity during structural changes **?**. These tools demonstrate that meaningful semantic structure can emerge from authoring conventions. However, they lack machine-checkable coordination primitives: no constraint language prevents schema drift, and no consensus mechanism mediates conflicting contributions from multiple agents. ADL Lite builds directly on this document-native paradigm while addressing each of these gaps.

2.2.1 LLM-Native Ontology Engineering

The past two years have witnessed the emergence of LLM-native approaches to ontology engineering, in which large language models serve not merely as assistive tools but as autonomous agents in the ontology construction pipeline. This subsection surveys three representative lines of work and positions ADL Lite relative to each.

LLM4ACOE ? is the first complete multi-agent collaborative ontology engineering framework. It uses LLMs to simulate three roles—Knowledge Engineer, Domain Expert, and Knowledge Worker—and employs retrieval-augmented generation (RAG) to inject domain data and OWL documents, generating domain ontologies automatically. While LLM4ACOE demonstrates that multi-agent LLM systems can produce coherent ontologies, it generates standard OWL without cryptographic provenance and provides no lifecycle governance mechanism for concepts.

Sim-HCOME ? extends the HCONE methodology with fully automated LLM role-playing, validating its approach on SAR tasks and Parkinson's disease ontologies. It reveals systematic patterns in how ontology quality changes as human involvement decreases, but like LLM4ACOE, it produces static OWL files without append-only audit trails.

Ontology-Constrained Neural Reasoning ? proposes a three-layer architecture—Role Ontology, Interaction Ontology, and Governance Constraints—that uses ontological constraints to limit the operational envelope of LLM agents. This work demonstrates that ontological governance can reduce agent hallucination, but the constraints are expressed in OWL and require a full reasoning stack.

A comprehensive survey by Garijo and Shimizu **?** analyzes over 200 papers and identifies five task categories for LLMs in ontology engineering: learning, enrichment, alignment, evaluation, and query generation. The survey notes that LLM-generated OWL axioms often suffer from insufficient expressiveness and lack provenance tracking.

These approaches collectively establish three research routes: (1) fully automated multi-agent collaboration producing OWL (LLM4ACOE, Sim-HCOME); (2) LLM-assisted traditional OE workflows (OntoGPT, Chowlk); and (3) ontology-constrained agent behavior (Ontology-Constrained Agents). ADL Lite occupies a fourth route: *document-native authoring with event-sourced provenance and cryptographic integrity*—a combination that no existing LLM-native OE framework has explored.

2.3 Provenance, Trust, and Verifiable Publishing

The landscape of Semantic Web provenance encompasses several standards that address complementary aspects of the problem space that ADL Lite targets: recording *who* said *what*, *when*, and with *what* evidentiary basis.

W3C PROV-O. The W3C PROV Ontology W3C [2013] provides a standard model for provenance interchange, expressing the PROV Data Model in OWL 2. PROV-O defines core concepts including `Entity`, `Activity`, and `Agent`, along with properties such as `wasGeneratedBy` and `wasAssociatedWith` that establish causal and responsibility relationships. ADL Lite's EventChain can be directly mapped to PROV-O: each Event corresponds to a `prov:Activity`, the concept being modified corresponds to a `prov:Entity`, and the `actor` field maps to `prov:wasAssociatedWith`. However, PROV-O alone does not provide cryptographic integrity guarantees—it is a data model for describing provenance, not a mechanism for verifying it. ADL Lite complements PROV-O by extending its descriptive provenance model with built-in SHA-256 hash chaining.

Nanopublications and Trusty URIs. The nanopublication framework Kuhn et al. [2015] provides a standardized way to publish small, granular scientific claims with cryptographically signed provenance. Each nanopublication consists of three named graphs: an assertion graph, a

provenance graph, and a publication-info graph **?**. The Trusty URI mechanism Kuhn and Dumontier [2015] provides content-addressed identifiers via Base64-encoded SHA-256 hashes, ensuring that any modification to a nanopublication's content changes its URI. This achieves decentralized verifiable publishing—a goal shared with ADL Lite's append-only EventChain. Table 2 provides an empirical comparison across nine operational dimensions.

Table 2: Empirical comparison of nanopublications with Trusty URIs and ADL Lite EventChains.

Dimension	**Nanopublications + Trusty URIs**	**ADL Lite EventChain**
Granularity	Atomic claim (single RDF triple or small graph)	Concept lifecycle (multi-event chain)
Mutability	Immutable: new version = new Trusty URI	Append-only: events accumulate, status derived
Authentication	RSA digital signatures per nanopub	String identifiers (Phase 1); Ed25519 LD-Proofs (Phase 3)
Integrity mechanism	Content-addressed SHA-256 hash in URI	Cryptographic hash chain (h_{prev} linkage)
Structural overhead	31–86% non-assertion triples	≈200 bytes/hash per event (<1% for typical chains)
Infrastructure	RDF/SPARQL triplestore + nanopub server	Markdown files in standard Git repositories
Verification time	$O(1)$ per nanopub (hash comparison)	$O(n)$ per chain (sequential hash verification)
Lifecycle governance	None (status is publication metadata)	Deterministic status derivation + precondition rules
Fork support	Manual: create new nanopub, no automatic linkage	Automatic: FORK event + derived child chain

Linked Data Proofs. The W3C Verifiable Credential Data Integrity specification W3C Verifiable Credentials Working Group [2024] describes mechanisms for ensuring authenticity and integrity of digital documents through digital signatures over canonicalized RDF datasets. Underpinning this is the RDF Dataset Canonicalization algorithm (URDNA2015) **?**, which produces a deterministic serialization regardless of original triple ordering. This technology provides authenticated provenance that ADL Lite currently lacks: the ability to bind actor identities to events through cryptographic signatures rather than string identifiers. ADL Lite's Phase 3 includes LD-Proof integration for authenticated events.

RDF-star. RDF-star W3C RDF-star Working Group [2024] provides statement-level annotation by enabling triples to function as subjects or objects of other triples, without the syntactic verbosity of explicit reification. ADL Lite's L3 relation blocks—encoding `source-relation-target` triples within Markdown fenced code blocks—can be viewed as a Markdown-native form of RDF triples, with each EventChain entry providing statement-level provenance analogous to an RDF-star annotation.

Blockchain-Based Provenance. Decentralized provenance systems based on blockchain technology (e.g., Ethereum + IPFS, Hyperledger Fabric for supply chain traceability, Verifiable Credentials on distributed ledgers) provide tamper-evident audit trails through consensus-based append-only ledgers. These systems share ADL Lite's commitment to cryptographic hash chaining and append-only data structures. However, they impose deployment costs (network fees, consensus protocol overhead, smart contract maintenance) that are unnecessary for small-team collaborative

governance. ADL Lite achieves comparable integrity guarantees at substantially lower operational cost by leveraging Git—a widely deployed, mature version control system with built-in signed-commit support—as its storage layer. The trade-off is that ADL Lite does not provide Byzantine fault tolerance (see Section 4.8), whereas permissionless blockchains do.

Git Signed-Commits as Operational Analogs. Git's signed-commit workflow (GPG-signing commits with developer keys) provides a close operational analog to ADL Lite's trust model: both systems rely on append-only history with cryptographic integrity. In Git, each commit is hashed (SHA-1 or SHA-256) and optionally signed; the commit history forms a Merkle-DAG. ADL Lite's EventChain extends this pattern from source code to knowledge artifacts, adding: (1) deterministic concept status derivation (δ), (2) confidence aggregation across multiple validators (γ), (3) structured precondition enforcement for lifecycle transitions, and (4) fork semantics that preserve lineage identity. Git provides the storage and transport; ADL Lite provides the semantic governance layer on top.

2.4 Event-Centric Ontologies

The ontologies surveyed above vary in their treatment of events. This subsection examines event-centric approaches that share ADL Lite's premise that *change in the world is best understood through events* rather than through direct property mutations of persistent objects.

2.4.1 CIDOC CRM

The CIDOC Conceptual Reference Model ICOM-CIDOC [2014] is an ISO 21127 standard for cultural heritage information integration. CIDOC CRM is fundamentally event-centric: it models reality through the `E5 Event` class that brings together `E39 Actor` participants with `E18 Physical Thing` objects in spatiotemporal contexts. The ontology's core commitment is that change is understood through events, not through property mutations. ADL Lite shares this philosophical stance: concept lifecycle changes are explicit events in an append-only chain rather than mutable field updates. The key difference is purpose: CIDOC CRM is *descriptive* (documenting what happened), while ADL Lite is *operational* (enforcing what can happen through preconditions).

2.4.2 SEM and LODE

The Simple Event Model (SEM) **?** and Linked Open Descriptions of Events (LODE) **?** represent lightweight alternatives for publishing event data on the Semantic Web. SEM provides a minimal vocabulary comprising `sem:Event`, `sem:Actor`, `sem:Place`, and `sem:Time`. LODE extends SEM with Dublin Core compatibility and SKOS-based event type hierarchies. Both are *descriptive* ontologies for event publishing: they enable data providers to expose event descriptions in RDF for consumption and querying. They do not provide constraint mechanisms for governing which events can occur, nor do they offer integrity guarantees for event sequences. ADL Lite complements these frameworks with an *operational* layer: the EventChain is a tamper-evident, precondition-governed mechanism for controlling concept lifecycle transitions.

2.4.3 RDF Stream Processing

The W3C RDF Stream Processing Community Group defined models for continuous processing over timestamped RDF triples arriving in streams **?**. Systems such as C-SPARQL Barbieri et al. [2009] and CQELS **?** extend SPARQL with windowing operators and continuous query semantics, enabling real-time pattern detection over streaming RDF data. ADL Lite's EventChain can be

viewed as a *materialized* RDF stream: each event appends triples to a persistent, cryptographically linked chain, enabling retrospective provenance queries rather than real-time pattern detection.

2.4.4 Subject-Event Ontology and Occurrence-Only Modeling

Two recent proposals share ADL Lite's commitment to event-first modeling but pursue different formal foundations.

Reactive Subject-Event Ontology (SEO). Boldachev **?** proposes a complete Subject-Event Ontology in which events are causal primitives connected by a happens-before ($\prec$) relation forming a directed acyclic graph. SEO supports reactive guards—triggers that fire when causal preconditions are met—and validates consistency through conformance tests against formal axioms (A1–A9) and invariants (I1–I3). The key structural difference from ADL Lite is topology: SEO uses a *causal graph* (events as nodes, $\prec$ as edges), while ADL Lite uses a *linear hash chain* (events as sequence elements, SHA-256 as linkage). The causal graph excels in distributed concurrent scenarios where multiple agents contribute events without global coordination; the linear chain excels in sequential audit scenarios where total order and cryptographic tamper-evidence are paramount. ADL Lite prioritizes the latter because concept lifecycle governance is inherently sequential: a concept must be `REGISTER`ed before it can be `VALIDATE`d, and the order of these transitions carries semantic weight that a partial order cannot capture.

Occurrence-Only Modeling. Al-Fedaghi **?** proposes an "occurrence-only" paradigm that models the world as compositions of events rather than as objects with properties. Using "region" (static blueprint) and "breath" (dynamic temporal span) as core categories, this work provides independent philosophical grounding for the event-first stance that ADL Lite adopts from Wittgenstein. While Al-Fedaghi's framework lacks cryptographic integrity mechanisms and multi-agent governance, its category system offers a complementary ontological analysis that could inform future extensions to ADL Lite's event taxonomy.

UFO-B Executable Specifications. Guizzardi et al. **?** extend UFO-B (the event-driven extension of UFO) with "Social Relator," "Disposition," and "Institutional Event" as executable event-log specifications. Events become the sole mechanism for changing social objects. ADL Lite's L4 Action blocks correspond directly to UFO-B "Institutional Events": both are intentional actions governed by preconditions that change the state of the ontological world. The key difference is formalization depth: UFO-B provides full axiomatic semantics in first-order logic, while ADL Lite provides lightweight YAML-declared preconditions validated at parse time.

2.4.5 Conflict-Free Replicated Data Types

CRDTs Shapiro et al. [2011] provide a theoretical foundation for reconciling concurrent updates without centralized coordination. Automerge Martin et al. [2020] is a JSON CRDT library that preserves all concurrent edits rather than choosing a single winner. Yjs Nicolai and Dadam [2021] is a widely deployed CRDT framework for real-time collaborative editing, optimized for shared text documents. Recent research has explored applying CRDTs to knowledge graph editing Anonymous [2023]. ADL Lite's Git-native design already provides branch-based concurrent authoring; what CRDTs would add is automatic merge semantics for event chains, eliminating the last-write-wins limitation without sacrificing tamper evidence.

Formally Verified CRDTs. Nieto **?** provides abstract specifications for CRDTs in the Iris separation logic framework, defining CRDT states as denotations of event sets and proving convergence properties. ADL Lite's EventChain can be viewed as a specific CRDT—an append-only log—but lacks the formal convergence guarantees that Iris provides. This gap represents a promising

direction for future formalization (see Section 7.2).

Byzantine-Tolerant CRDTs. The Blocklace **?** proposes a cryptographic hash DAG as a universal CRDT capable of detecting and excluding Byzantine nodes. ADL Lite's EventChain is a linear special case of the Blocklace DAG, but ADL Lite currently lacks Byzantine fault tolerance—a limitation scoped to Phase 1's collaborative-audit threat model. The Blocklace's equivocation-detection mechanism could inform Phase 3's transition to adversarial robustness.

Concurrent Admin Conflict Resolution. Recent work on epoch-resolved arbitration **?** addresses the "duelling admins" problem in CRDT-based group management, where concurrent revocation operations create unresolvable conflicts. The proposed epoch mechanism—assigning monotonic epoch numbers to administrative operations and using epoch comparison to break ties—offers a concrete path beyond ADL Lite's current last-write-wins semantics for fork resolution.

PLUGMEM – Agent Memory with Governance Gap. PLUGMEM (arXiv:2603.03296) is a recent agent memory framework that provides structured memory abstraction and retrieval for LLM agents. PLUGMEM acknowledges that it lacks governance and versioning mechanisms for agent memories—a gap that ADL Lite directly addresses. ADL Lite's EventChain model could serve as a governance substrate for PLUGMEM's memory entries: each memory snapshot becomes a concept whose lifecycle (creation, validation, deprecation) is governed by the EventChain, providing the provenance and audit trail that PLUGMEM currently lacks. This integration represents a complementary pairing: PLUGMEM handles *memory storage and retrieval*, while ADL Lite handles *lifecycle governance and consensus* of the knowledge encoded in those memories. Demonstrating such integration is planned for future work.

2.4.6 Summary: Event Topology and Integrity Mechanisms

Table 3 compares ADL Lite with the event-centric frameworks surveyed above across three salient dimensions: event topology (how events are structured), integrity mechanism (how tampering is detected), and governance semantics (how lifecycle transitions are controlled).

Table 3: Comparison of event-centric frameworks: topology, integrity, and governance.

Framework	**Event Topology**	**Integrity Mechanism**	**Governance Semantics**
CIDOC CRM	Descriptive event graph	None (documentation)	None
SEM/LODE	Minimal event vocabulary	None (publishing)	None
SEO (2025)	Causal DAG ($\prec$ relation)	Conformance tests (A1–A9)	Reactive guards
Blocklace (2024)	Cryptographic hash DAG	SHA-256 + BFT consensus	Byzantine exclusion
ADL Lite	Linear SHA-256 chain	Sequential hash verification	Precondition rules + $\delta(C)$

2.5 Positioning

Palantir Foundry represents the most prominent industrial embodiment of an operational ontology platform. Its Data Engine exposes four core ontological primitives: Object Type, Property, Link,

and Action Palantir Technologies [2024]. This architecture unifies "semantic elements" (nouns) with "kinetic elements" (verbs), enabling AI agents to both query and act upon the ontology through governed interfaces Palantir Technologies [2024]. Despite its sophistication, Palantir exhibits three limitations: enterprise-scale deployment cost, proprietary ontology authoring tooling, and no design for LLM-native authorship. ADL Lite addresses each by offering pip-installable deployment, Markdown-native authoring, and explicit multi-agent concept consensus.

Table 4 positions ADL Lite against three reference approaches across seven salient dimensions.

Table 4: Positioning of ADL Lite against reference approaches.

Dimension	**Palantir FDE**	**OWL/SPARQL**	**Markdown-only**	**ADL Lite**
Deployment Cost	Enterprise ($M+$)	High (triple store + reasoner)	Low (Git repo)	Very Low (`pip install` + Git)
LLM Authorship	Not designed	Requires ontology expertise	Informal (free-text)	Native (YAML + Markdown)
Integrity Guarantees	Platform-managed	Reasoner-based consistency	None	SHA-256 EventChain (append-only)
Action Preconditions	TypeScript functions	OWL/SWRL rules	None	Declarative YAML + SSA validation
Consensus Mechanism	Centralized (admin UI)	N/A (single authority)	N/A (no coordination)	Multi-agent quorum + event history
Scalability	Billions of objects	Millions of triples	Thousands of notes	Pilot: 201 chains; target: 10^4
Standards Alignment	Proprietary	Full W3C stack	None	Partial (PROV-O mappable, SHACL-integrable)

ADL Lite occupies a deliberate middle ground. Palantir delivers industrial-strength operational semantics at deployment cost and accessibility barriers that exclude small teams and LLM-native workflows. OWL/SPARQL pipelines offer unmatched formal expressiveness yet require specialized expertise and infrastructure. Markdown-based tools democratize knowledge capture but provide no machine-checkable guarantees against schema drift or conflicting definitions.

The gap that ADL Lite fills is the intersection of four properties that no existing approach combines: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. PROV-O provides (a) only when combined with RDF authoring tools and lacks (b) natively; nanopublications provide (b) for published claims but lack (a), (c), and (d); CIDOC CRM provides event-centric descriptive modeling (ISO 21127) but, when deployed with full RDF/OWL toolchains for reasoning-enabled applications, benefits from triple stores and SPARQL endpoints that may exceed the infrastructure budget of

small teams. ADL Lite enables multiple LLM agents proposing, challenging, and refining concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that makes decentralized authorship accountable.

3 Ontological Analysis of ADL Lite

3.1 Philosophical Foundation: Event-First as Ontological Commitment

The architectural design of ADL Lite is grounded in an ontological inversion of conventional ontology engineering. Traditional operational ontologies—including OWL-based knowledge graphs, Palantir Foundry, and enterprise metadata platforms—adopt an *object-first* stance: an Object Type defines what entities exist, a Property Type describes their attributes, a Link Type connects them, and an Action Type is appended as an operational afterthought to model operations upon the previously defined object structures Palantir Technologies [2024], **?**. This ordering reflects a metaphysical commitment to the *primacy of continuants*: entities are conceived as self-subsistent bearers of properties that persist through changes in their attributes.

ADL Lite inverts this hierarchy. Drawing on Wittgenstein's *Tractatus Logico-Philosophicus* §1.1—*"Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge"* ("The world is the totality of facts, not of things")—the system treats the event as the fundamental ontological primitive Wittgenstein [1922], Young [2010]. On this view, an object has no ontological standing independent of the facts in which it appears. A concept's status (`validated`, `deprecated`, `forked`) is not an intrinsic property of the concept but a relational property constituted by the history of events that have occurred in its lifecycle.

This philosophical commitment translates into three governing ontological principles:

(1) **No implicit state changes.** Every mutation of ontological state is captured as an explicit event. There are no "hidden" updates, no side-effectful assignments, and no mutable fields that can diverge from their event histories.

(2) **Events are occurrents.** Events are temporal entities that happen; they do not endure through time. An event has no spatial extent and no existence outside the interval at which it occurs. This aligns with the BFO category of *occurrent* **?** and DOLCE's category of *perdurant* **?**.

(3) **All derived properties are historical.** Status, confidence, and validator identity are computed by deterministic functions over the event sequence. They are not stored; they are re-derived on demand. This eliminates an entire class of consistency bugs in which stored state diverges from logical history.

3.2 Categories and Taxonomy

ADL Lite's category system can be analyzed through the lens of three influential upper ontologies: Basic Formal Ontology (BFO) **?**, Descriptive Ontology for Linguistic and Cognitive Engineering (DOLCE) **?**, and the Unified Foundational Ontology (UFO) **?**. Table 5 presents the cross-mapping. The following subsections discuss each category in detail and justify alignment choices.

3.2.1 Event as Occurrent/Perdurant

In BFO, an *occurrent* is an entity that unfolds itself in time. Occurrents include processes (e.g., a cell division), and they are ontologically distinct from *continuants* (e.g., a cell), which exist in

Table 5: Cross-mapping of ADL Lite categories to upper ontologies. Entries marked "—" indicate intentional non-alignment; entries marked "≈" indicate approximate correspondence.

ADL Lite	**BFO 2.0**	**DOLCE**	**UFO**	**N**
Event	`occurrent`	`perdurant`	`Event`	T
EventChain	`process`	`accomplishment≈`	`Complex Event`	O
Concept	`generically dependent continuant`	`non-physical object≈`	`Conceptual Entity`	D
Relation (L3)	`relational quality`	`quality`	`Relator`	M
Action (L4)	`planned process`	`action`	`Action`	In
Actor	`agent`	`physical agent`	`Agent`	C
Payload	`information content entity`	`information object`	`Information Entity`	St
Hash	`generically dependent continuant`	—	—	C

full at every moment of their existence **?**. In DOLCE, the corresponding category is *perdurant*: an entity that is only partially present at any time, in the sense that some of its temporal parts may be absent **?**.

ADL Lite's `Event` is a perdurant in the DOLCE sense and an occurrent in the BFO sense. It is a temporal entity with a definite beginning and end (the timestamp). It has no spatial location (unless the payload carries one, which is domain-specific). It does not endure: once a `VALIDATE` event has occurred, it is complete; it does not continue to exist as a present entity. What endures is the *record* of the event (the serialized JSON/YAML in the Markdown file), but this record is an *information content entity*, not the event itself.

In UFO, events are also treated as perdurants, but UFO additionally distinguishes *events* (changes in the world) from *actions* (intentional events performed by agents) **?**. ADL Lite's event type system captures this distinction: `REGISTER`, `VALIDATE`, `DEPRECATE`, `FORK`, `ARCHIVE` are actions (intentional, performed by an actor), while `RELATE`, `EVIDENCE`, `SEAL` are events (assertions that change the relational structure of the concept graph).

3.2.2 Concept as Generically Dependent Continuant

The ontological status of a "concept" in ADL Lite is subtle. A concept is not a physical object (it has no mass, no spatial location). It is not a mental object (it exists outside any individual mind, in the shared Markdown file). It is not a universal (it is not instantiated by particulars). We propose that an ADL Lite concept is best understood as a *generically dependent continuant* in BFO terms **?**: an entity that depends on some continuant or other for its existence, but not on any *specific* continuant.

A concept depends generically on its EventChain *as an information content entity (ICE)*—a continuant that encodes the event history—rather than on the occurrent process of events happening. In BFO 2020, an ICE is a GDC concretized by some material bearer (the Markdown file on disk). The events themselves are occurrents; the serialized chain record is the continuant ICA that bears the concept's identity.[1] If the chain record were destroyed, the concept would cease to exist. But the concept does not depend on any specific event within the chain—events can be added and the concept persists. The concept's identity is determined by the *genesis event* (the first event in the chain, with its `concept_id`), not by the totality of events. This is analogous to how a BFO

[1]This distinction resolves an apparent category mismatch: a GDC must depend on a continuant bearer, not on an occurrent. The EventChain qua ICE is the bearer; the events occurring constitute the temporal process that produces and updates the ICE.

generically dependent continuant such as a poem depends on some physical copy for its existence, but not on any specific copy.

In DOLCE, the closest category is *non-physical object* (NPO), which includes social objects, information objects, and cognitive objects **?**. ADL Lite concepts are information objects that carry social/institutional weight (their status determines whether they are accepted by the community). In UFO, concepts are *conceptual entities* or *types* in the conceptual space of the community **?**.

3.2.3 EventChain as Process

An EventChain is a sequence of events. In BFO, it is best understood as a *process*: a complex occurrent that has temporal parts (the individual events) and unfolds over time **?**. The chain is not a continuant because it does not exist in full at any single moment of time—at time t, only the events up to t have occurred.

This has implications for identity conditions. In BFO, a process is individuated by its constituent processes and their temporal relations. An EventChain is individuated by the ordered sequence of its events and their cryptographic linkage. Two chains with the same events in the same order are the same chain (modulo the `concept_id`, which anchors the chain to a specific concept).

3.2.4 Relation as Relational Quality / Relator

ADL Lite's L3 relation blocks (`adl:relation`) assert typed edges between concepts: `source -relation-> target`. In BFO, this is best analyzed as a *relational quality*: a quality that inheres in one entity and relates it to another **?**. For example, the relation "isomorphic-to" between two concepts is a relational quality that inheres in the source concept and points to the target concept.

In UFO, the corresponding category is *relator*: an entity that mediates the relationship between two or more entities **?**. A relator exists because the related entities exist, and it is destroyed when the relationship ceases to hold. In ADL Lite, a relation is established by a `RELATE` event and can be revoked (in future work) by a `UNRELATE` event. This event-based lifecycle aligns with the UFO notion that relators are brought into existence and destroyed by events.

DOLCE's category of *quality* is broader and includes both intrinsic qualities (e.g., color) and relational qualities (e.g., being taller than) **?**. ADL Lite relations are relational qualities in the DOLCE sense.

3.2.5 Action as Planned Process / Intentional Event

ADL Lite's L4 action types (`REGISTER`, `VALIDATE`, etc.) are actions in the full philosophical sense: they are intentional events performed by agents for reasons. In BFO, they are *planned processes*: processes that realize a plan **?**. In UFO, they are *actions*: intentional events performed by agents to achieve a goal **?**.

The precondition system in ADL Lite captures the *plan* aspect: an action can only be performed if certain conditions hold (e.g., `VALIDATE` requires `status == provisional`). This is not merely a computational check; it is an ontological commitment to the idea that actions are governed by norms that constrain which events can occur in which states.

3.3 Identity Conditions

Identity conditions specify when two entities are the same. ADL Lite requires identity conditions for three categories: events, concepts, and chains.

3.3.1 Event Identity

An event e is identical to an event e' if and only if:

$$e = e' \iff e.\text{event\_id} = e'.\text{event\_id} \tag{1}$$

The `event_id` is a UUID generated at event creation time. It is independent of the event's content, timestamp, or chain position. This means that two events with identical content but different `event_id`s are *distinct events* (they occurred at different times or in different contexts). This aligns with the intuition that two validation events by the same actor at different times are different events, even if their payloads are identical.

The cryptographic hash e.hash is *not* an identity criterion; it is an integrity criterion. Two events with the same hash have the same content and linkage, but they may still be different events (if they have different `event_id`s, which is impossible because the `event_id` is in the hash input). In practice, the hash uniquely determines the event's content, but the `event_id` is the canonical identifier.

3.3.2 Concept Identity

A concept c is identical to a concept c' if and only if:

$$c = c' \iff c.\text{concept\_id} = c'.\text{concept\_id} \tag{2}$$

The `concept_id` is a human-readable identifier (e.g., `disc-capital-trap`) assigned at registration time. It is independent of the concept's status, confidence, or event history. This means that a concept retains its identity even if all its events are forked, deprecated, or archived.

This choice reflects an ontological commitment to *historical identity*: a concept is the same concept throughout its lifecycle because it has the same genesis event (the `REGISTER` event that created it). The `concept_id` is a rigid designator for the concept, analogous to how a proper name rigidly designates an individual in modal logic.

A forked concept (e.g., `disc-capital-trap-v2`) is a *different concept* because it has a different `concept_id`, even though it is derived from the original. This is analogous to how a biological daughter is a different organism from her mother, even though she derives from her mother's genetic material.

3.3.3 EventChain Identity

An EventChain C is identical to an EventChain C' if and only if:

$$C = C' \iff C.\text{concept\_id} = C'.\text{concept\_id} \wedge \text{events}(C) = \text{events}(C') \tag{3}$$

Two chains are the same if they have the same `concept_id` and the same ordered sequence of events. The cryptographic linkage ensures that the event sequence is ordered, so two chains with the same events in different orders are different chains.

3.4 Ontological Dependence

Ontological dependence concerns which entities depend on which others for their existence. ADL Lite exhibits three forms of dependence:

3.4.1 Identity and Dependence Conditions

The treatment of Concept as a generically dependent continuant (GDC) that depends on an EventChain raises questions about cross-category dependence that merit precise formulation. We formalize the dependency as:

$$\mathsf{Concept} \xrightarrow{\text{GDC}} \mathsf{ICE(EventChain\_record)} \xrightarrow{\text{concretized\_by}} \mathsf{Markdown\_file}$$

The Concept depends on an *information content entity* (ICE)—the serialized EventChain record—which is itself a GDC concretized by the Markdown file on disk. The events occurring in the chain are occurrents; the Concept does not depend on them directly, but on the ICE that records them. This resolves the apparent category mismatch: a GDC cannot depend on an occurrent, but it can depend on the ICE that documents that occurrent.

What does the Concept depend on? A Concept in ADL Lite depends on the *information artifact* (the serialized EventChain record) rather than on the occurrent (the process of events occurring). This is consistent with BFO's GDC criteria: a GDC depends on some continuant bearer but is not identical to any particular bearer. The EventChain is the bearer—a structured information entity that encodes the history of occurrents. The Concept (as GDC) is the *interpretation* of that history: the rules that project status, confidence, and validators from the event sequence.

Why not depend on the occurrent directly? In BFO, GDCs cannot depend on occurrents because occurrents lack the stability required for generic dependence. ADL Lite respects this constraint by making the Concept depend on the *record* of events (an information content entity, which is a continuant), not on the events themselves (occurrents). The EventChain is a generically dependent continuant in its own right: it exists in virtue of being encoded in some physical medium (file, database), but not in virtue of any *specific* medium.

Identity conditions. A Concept's identity is determined by its genesis event—the `REGISTER` event that first introduces the `concept_id`. This aligns with BFO's identity criteria for GDCs: identity is conferred by the process of creation (the genesis event), not by the current state of the bearer. Two EventChains with the same `concept_id` and genesis event hash represent the same Concept, even if their subsequent events diverge (in which case they are *forks* of the same Concept, each carrying the original identity into a distinct lineage).

Dependence on genesis event. The dependence relation is not merely historical but *constitutive*: without the genesis event, there is no Concept, because there is no `concept_id` to serve as the rigid designator. This is analogous to how a novel depends on its first publication: subsequent editions may vary, but the identity of the work is fixed by the original publication event.

Multiple concretizations. As a GDC, a Concept can be concretized in multiple copies. An EventChain may be replicated across several Git repositories, local filesystems, and agent caches simultaneously. In BFO terms, each copy is a distinct *concretization* of the same ICE, and therefore the same Concept. The identity of the Concept is determined by the *information content* of the genesis event (specifically, the SHA-256 hash of the genesis event's canonical serialization), not by which specific copy is accessed. Two agents with independent copies of the same EventChain refer to the same Concept, because the genesis event hash is identical across copies. This parallels how a poem (GDC) exists in multiple printed books: each book is a different concretization, but the poem is the same work.

Identity persistence across re-materialization. If all copies of an EventChain are lost (e.g., Git repository deleted) but later restored from a backup or reconstructed from a peer agent's copy, the Concept's identity persists. The restored copy has the same genesis event hash, and therefore the same `concept_id`. In BFO terms, the GDC (the Concept) continues to exist because at least

one concretization exists. Identity is robust to gap periods where no concretization is accessible, provided that at least one copy is recoverable. This is consistent with how a lost manuscript, if rediscovered, restores the existence of the work it instantiates.

Cessation of existence. A Concept ceases to exist in the BFO sense when and only when: (i) all concretizations (copies of the EventChain across all repositories and agent caches) are irreversibly destroyed with no possibility of recovery; and (ii) no agent retains sufficient information to reconstruct the EventChain (i.e., the genesis event hash and the full event sequence cannot be recovered from any source). Condition (ii) is extremely restrictive in practice: a single agent's memory of the genesis event hash is sufficient to validate any subsequently recovered copy. This is a high bar for cessation—intentional rather than accidental—and reflects the durability inherent in distributed, hash-addressed information systems.

3.4.2 Historical Dependence of Concept on EventChain

A concept c *historically depends* on its EventChain C: c would not exist if C had not been created. This is a form of *rigid existential dependence* **?**: the concept cannot exist without the chain. The dependence is historical because the concept's identity is determined by the genesis event in the chain.

This dependence is asymmetric: the chain can exist without the concept (in the trivial sense that the chain is the concept's history). But the concept cannot exist without the chain because the concept *is* its history.

3.4.3 Generic Dependence of Status on Events

A concept's status s (e.g., `validated`) *generically depends* on the events in its chain: s depends on there being at least one `VALIDATE` event, but not on any *specific* `VALIDATE` event. If a different agent had performed the validation at a different time, the status would still be `validated`. This is analogous to how a BFO generically dependent continuant depends on some bearer but not on any specific bearer.

3.4.4 Mutual Dependence of Relations on Concepts

A relation r between concepts c_1 and c_2 *mutually depends* on both concepts: r cannot exist if either c_1 or c_2 ceases to exist. This is a form of *mutual rigid dependence* **?**. If `concept-A` is deprecated and removed from the corpus, the relation "`concept-A` isomorphic-to `concept-B`" ceases to hold, even if `concept-B` still exists.

3.5 Comparison with Upper Ontologies: Alignment Choices and Deviations

Table 5 reveals three intentional deviations from standard upper-ontology practice:

Deviation 1: No distinction between continuants and occurrents at the language level. In BFO and DOLCE, the distinction between continuants and occurrents is a *language-level* distinction: the ontology language (OWL, FOL) enforces that continuants cannot be subclasses of occurrents. ADL Lite does not enforce this distinction at the language level; it is a *conceptual* distinction that is enforced by the event model. A "concept" is treated as a continuant in some contexts (it has an identity that persists through time) and as an occurrent in others (its state is computed from events). This flexibility is deliberate: it allows the same entity to be viewed under different ontological descriptions without requiring multiple representations.

Deviation 2: No explicit quality hierarchy. BFO and DOLCE both have elaborate hierarchies of qualities (intrinsic, relational, qualitative, etc.). ADL Lite does not model qualities explicitly; instead, qualities are encoded as payload fields in events. For example, "confidence" is not a quality that inheres in the concept; it is a value carried by `VALIDATE` events. This simplifies the ontology but loses the ability to reason about qualities independently of events.

Deviation 3: Actions as first-class citizens, not processes. In BFO, actions are a subclass of processes. In ADL Lite, actions are *co-equal* with objects at the language level: the action registry is defined before the object hierarchy. This reflects the event-first stance: actions are not operations *on* objects; they are the fundamental units of ontological reality, and objects are projections of event patterns.

These deviations are not arbitrary simplifications; they are motivated by the target application domain (multi-agent concept governance). In a domain where agents create, validate, and deprecate concepts at high velocity, the action-centric view provides a more natural modeling framework than the object-centric view. The cost is that ADL Lite cannot directly import BFO or DOLCE modules without translation; the benefit is that the ontology is optimized for its intended use case.

4 Architecture and Formal Semantics

4.1 Document Model: L1–L4 as Conceptual Modeling Layers

ADL Lite organizes concept documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 6 summarizes the document model.

Table 6: The L1–L4 Document Model. Each layer contributes distinct semantics to the concept document.

Layer	Syntax	Role	Event Type
L1	YAML front matter	Identity metadata (derived snapshot)	SNAPSHOT
L2	Markdown body	Human/LLM narrative prose	—
L3	`adl:relation`, `adl:evidence`, `adl:seal`	Typed semantic assertions	RELATE, EVIDE
L4	`adl:action`	Typed actions with preconditions	REGISTER, VAL

The L1 layer comprises YAML front matter containing identity metadata: `adl_type`, `adl_id`, `status`, `confidence`, `novelty`, `scope`, and `provisional_names`. Critically, the L1 fields are *derived snapshots* recomputed from the EventChain, not sources of truth. A discrepancy between L1 `status` and the chain-derived status indicates an incomplete event history in the source document.

The L2 layer consists of the Markdown body: human- and LLM-readable prose. The Singular Subjectivity Assertion (SSA) prohibits fuzzy referents—including pronouns such as "this," "that," "it"—that would compromise cross-agent alignment **?**. SSA is configurable: `strict` mode enforces explicit reference for all agent-authored content, while `lax` mode permits natural pronoun use for human-authored narrative.

The L3 layer introduces typed semantic assertions through fenced code blocks: `adl:relation` for typed edges between concepts (supporting predicates such as `isomorphic-to` and `specialisation-of`), `adl:evidence` for evidentiary support, and `adl:seal` for formal assertions. These blocks are machine-parseable RDF-like triples that remain editable in ordinary text editors.

4.1.1 L3 Relation Lifecycle Operations

ADL Lite's L3 relation blocks support three lifecycle operations: *establishment*, *modification*, and *revocation*. Each operation is recorded as an event in the concept's EventChain, enabling full provenance tracking for relational assertions.

Establishment. A relation is established by a `RELATE` event with fields: `source`, `relation` (predicate from closed registry), `target`, optional `confidence`, and optional `mapping_type`. The relation becomes part of the concept's derived state. Precondition: the `relation` predicate must be registered in `adl_core_ontology.yaml` (strict mode) or will be accepted with a warning (relaxed mode).

Modification. A relation's confidence or mapping type can be modified by a subsequent `RELATE` event with the same `source`, `relation`, and `target` but different `confidence` or `mapping_type`. The new event *supersedes* the previous one: derived queries return the most recent value. Both events remain in the chain for audit purposes. This is analogous to SQL UPDATE with full audit logging.

Revocation. A relation is revoked by a `RELATE` event with the same `source`, `relation`, `target` but `confidence = 0.0` or a special `revoked: true` flag. The relation is excluded from derived state. The revocation itself is cryptographically linked to the original establishment event via the chain, providing tamper-evident revocation provenance.

Truth maintenance under forks. When a concept forks, all active relations from the parent chain are *inherited* by the child chain at the fork point. Subsequent modifications in either lineage do not affect the other—each fork maintains its own relation state. This is consistent with the fork semantics: a fork is a competing interpretation, and competing interpretations may have different relational assertions about the same entities.

Conflicting relation assertions. If two agents assert contradictory relations (e.g., A says "X isomorphic-to Y" with confidence 0.9, B says "X unrelated-to Y" with confidence 0.8), both events are appended to the chain. The derived state includes both assertions (they are not mutually exclusive at the event level), and downstream consumers must apply their own resolution logic. ADL Lite provides the *provenance infrastructure* for conflict detection but does not prescribe a single resolution strategy—this is a deliberate design choice to accommodate diverse domain requirements.

The L4 layer introduces `adl:action` blocks: typed actions with structured precondition rules that govern lifecycle transitions and trigger side effects. The four-layer design enables a single Markdown file to serve simultaneously as human-readable documentation, LLM authoring surface, machine-parseable semantic assertion container, and auditable action log.

4.2 EventChain: Core Data Structure

The EventChain is the fundamental data structure of ADL Lite. Each ontological concept is represented by exactly one EventChain: an append-only sequence of cryptographically linked events.

An *event* e is a structured record comprising nine fields: `event_id` (UUID-derived unique identifier), `concept_id` (the concept to which the event belongs), `event_type` (drawn from a closed set of lifecycle, assertion, and communication types), `actor` (the entity causing the event), `reasoning` (justification), `timestamp` (ISO-8601 occurrence time), `payload` (event-specific key-value data), `previous_event_id` (cryptographic link to the preceding event), and `hash` (SHA-256 digest of serialized content).

The event type system is partitioned into two disjoint sets:

$$\Sigma_{\text{life}} = \{\text{REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE}\} \tag{4}$$

$$\Sigma_{\text{comm}} = \{\text{RELATE, EVIDENCE, SEAL, ANNOUNCE, PUBLISH, SNAPSHOT}\} \tag{5}$$

Lifecycle events affect concept status; communication events do not. This partition is central to the derivation semantics (Section 4.6).

4.3 Canonical Serialization and Cryptographic Integrity

Each event's hash is computed over a canonical JSON serialization with the following properties: (i) keys are sorted lexicographically; (ii) floating-point values are rounded to six decimal places; (iii) all fields including the timestamp are included in the serialization; (iv) the hash field itself is excluded to avoid circular self-reference.

Timestamps are *included* in the hash input to ensure that two otherwise-identical events created at different times produce different hashes, preserving tamper-evidence for provenance. Event *ordering* within a chain uses cryptographic linkage (`previous_event_id`) rather than timestamp comparison, so clock skew does not affect chain validity.

The integrity verification predicate VerifyIntegrity(C) checks three conditions for a chain $C = [e_1, \ldots, e_n]$:

(1) Genesis anchoring: e_1.previous_event_id = null and e_1.prev_hash = genesis.

(2) Cryptographic linkage: for all $i \in \{2, \ldots, n\}$, e_i.prev_hash = e_{i-1}.hash.

(3) Hash correctness: for all $i \in \{1, \ldots, n\}$, e_i.hash = SHA-256(Canon(e_i)).

4.4 Action Executor with Structured Preconditions

The ActionExecutor validates L4 action blocks against a closed action registry defined in `adl_core_ontology.yam`
Each action definition specifies preconditions as a list of `PreconditionRule` objects, each comprising a `field` (the ADLFrontMatter field to inspect), a `comparator` (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS), and a `value` (the expected value or threshold).

Precondition evaluation uses explicit lambda dispatch on the `Comparator` enumeration; there is no runtime `eval()` or code injection path. This closed evaluation model achieves perfect precision and recall on all registered actions (Section 5.4).

4.5 Consensus and Status Transitions

Status transitions are governed by a transition matrix that defines which lifecycle events are permitted from each status. The matrix is total: every status has at least one permissible outgoing transition. The ConsensusEngine enforces these transitions by validating preconditions before appending events to chains.

Fork semantics deserve special attention. When a concept forks, a `FORK` event is appended to the original chain, and a new chain is created with a fresh `REGISTER` event. The original concept's status becomes `forked`; the new concept's status is `provisional`. Both chains evolve independently thereafter. This preserves the historical integrity of the original concept while enabling divergent evolution.

4.6 Formal Derivation Semantics

This section provides the complete formal foundation for the derivation semantics. All definitions are self-contained; expanded proof sketches appear in Appendix E. Table 7 summarizes the key notation.

Table 7: Formal notation for the derivation semantics.

Symbol	Definition
$\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$	Event alphabet (lifecycle ∪ communication)
$C = [e_1, \ldots, e_n]$	Well-formed EventChain
$\text{WF}(C)$	Well-formedness predicate (5 axioms)
$\delta(C) \in S$	Status derivation function
$\gamma(C) \in [0, 1]$	Confidence aggregation function
C_{life}	Lifecycle subsequence $\{e \in C \mid e.\tau \in \Sigma_{\text{life}}\}$
S	Status space {provisional, validated, deprecated, forked, archived}
$f : \Sigma_{\text{life}} \to S$	Event-type-to-status mapping
$\text{Fork}(C, a)$	Fork operation producing (C_{fork}, C')

4.6.1 Event Alphabet and Well-Formedness

Let $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ be the event alphabet. An event is a tuple $e = (\tau, a, t, p, h, h_{\text{prev}}, e_{\text{prev}})$ where $\tau \in \Sigma$, $a \in \mathcal{A}$ (actor set), $t \in \mathbb{R}_{\geq 0}$ (timestamp), $p \in \mathcal{P}$ (payload), $h \in \{0,1\}^{256}$ (hash), $h_{\text{prev}} \in \{0,1\}^{256} \cup \{\text{genesis}\}$, and $e_{\text{prev}} \in \text{ID} \cup \{\text{null}\}$.

A chain $C = [e_1, \ldots, e_n]$ is well-formed, denoted $\text{WF}(C)$, if:

(1) $e_1.h_{\text{prev}} = \text{genesis}$ and $e_1.e_{\text{prev}} = \text{null}$.

(2) All events in C share the same concept_id.

(3) For all $i \geq 2$: $e_i.h_{\text{prev}} = e_{i-1}.h$ and $e_i.e_{\text{prev}} = e_{i-1}.\text{event\_id}$.

(4) For all i: $e_i.h = \text{SHA-256}(\text{Canon}(e_i))$.

(5) All event_id values in C are pairwise distinct.

The canonicalization function $\text{Canon}(e)$ serializes e as UTF-8 JSON with sorted keys, six-decimal float rounding, and all fields (including timestamp) included.

4.6.2 Status Derivation δ

The status space is $S = \{\text{provisional, validated, deprecated, forked, archived}\}$. For a chain C, let $C_{\text{life}} = [e \in C \mid e.\tau \in \Sigma_{\text{life}}]$ be the lifecycle subsequence. Then:

$$\delta(C) = \begin{cases} \text{provisional} & \text{if } C_{\text{life}} = [] \\ f(\tau_{\text{last}}) & \text{otherwise} \end{cases} \tag{6}$$

where τ_{last} is the event type of the last element of C_{life}, and f maps REGISTER → provisional, VALIDATE → validated, DEPRECATE → deprecated, FORK → forked, ARCHIVE → archived.

4.6.3 Confidence Aggregation γ

Let $V = [e \in C \mid e.\tau = \text{VALIDATE}]$. Then:

$$\gamma(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \min(1.0,\ c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1)) & \text{otherwise} \end{cases} \tag{7}$$

where $\text{actors}(V) = \{e.a \mid e \in V\}$, $\phi(a, V) = \max\{e.p.\text{confidence} \mid e \in V \wedge e.a = a\}$, $c_{\text{base}} = \max(0.5,\ \text{mean}_{a \in \text{actors}(V)}\phi(a, V))$, and $N_{\text{vals}} = |\text{actors}(V)|$.

The per-actor maxima ensure duplicate validations from the same actor do not compound. The base confidence floor of 0.5 ensures that any validated concept has at least moderate confidence. The bonus of 0.05 per additional distinct validator rewards independent confirmation.

4.6.4 Fork and Confluence Semantics

A fork operation $\text{Fork}(C, a)$, parameterized by an initiating agent a, transforms a concept chain C into a parent-child pair:

$$\text{Fork}(C, a) = (C_{\text{fork}}, C') \tag{8}$$

$$C_{\text{fork}} = C \oplus [e_{\text{FORK}}] \tag{9}$$

$$C' = [e'_{\text{REG}}] \tag{10}$$

where e_{FORK} carries payload $\{\text{fork\_target} : C'.\text{concept\_id}, \text{reason} : \dots\}$ and e'_{REG} initialises the new concept with a fresh concept_id.

Fork-level confluence. For any well-formed C and any pair of agents a_1, a_2:

$$\text{Fork}(C, a_1)_{\text{child}} \perp \text{Fork}(C, a_2)_{\text{child}}$$

where $\perp$ denotes independence: each child chain's derived state depends only on its own event sequence, never on the sibling chain. This follows from the per-chain semantics of δ and the fact that child chains carry distinct concept_id values. The parent chain's status becomes forked regardless of which agent initiated the fork, preserving determinism (Theorem 1).

Optional CRDT merge semantics. For concurrent modifications by different agents who subsequently reconcile, ADL Lite supports optional LWW-Element-Set (LWW-Set) merge semantics on the confidence aggregation. For concurrent branches B_1, B_2:

$$\text{Merge}(B_1, B_2) = B_1 \cup B_2 \quad \text{(event union, deduplicated by event\_id)} \tag{11}$$

$$\text{with tie-breaking by} \quad \text{event\_id for equal-timestamp events} \tag{12}$$

The merge operation is commutative, associative, and idempotent, satisfying the CRDT convergence conditions: $\text{Merge}(B_1, B_2) = \text{Merge}(B_2, B_1)$ and $\text{Merge}(\text{Merge}(B_1, B_2), B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3))$.

We emphasise that CRDT merge is an *optional* strategy, not the default. The primary mechanism for handling divergent interpretations is fork-and-deprecate: competing views coexist as independent chains, and the community converges through social consensus—deprecating abandoned forks, validating accepted ones. ADL Lite provides the provenance infrastructure for both approaches without prescribing one over the other.

Theorem 1 (Determinism). For any well-formed C, $\delta(C)$ is unique and depends only on the last lifecycle event in C.

Proof. By definition, δ filters C to C_{life} (deterministic), checks emptiness (deterministic), and applies total function f to the last element (unique for ordered sequences). □

Theorem 2 (Confluence under Fork). Let C fork to (C_{fork}, C'). Then $\delta(C_{\text{fork}}) = \text{forked}$ and $\delta(C') = \text{provisional}$.

Proof. $e_{\text{FORK}} \in \Sigma_{\text{life}}$ is the last lifecycle event in C_{fork}, so $f(\text{FORK}) = \text{forked}$. C' contains only e'_{REG}, so $f(\text{REGISTER}) = \text{provisional}$. □

Theorem 3 (Transition Monotonicity). For any well-formed C and event e_{new}, the transition from $s = \delta(C)$ to $s' = \delta(C \oplus [e_{\text{new}}])$ is valid iff $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$.

Proof. If $e_{\text{new}}.\tau \notin \Sigma_{\text{life}}$, then C_{life} is unchanged, so $s' = s$ (identity transition, always valid). If $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\tau)$, which is exactly the target state defined by the transition matrix. □

Theorem 4 (Confidence Boundedness). For any well-formed C, $\gamma(C) \in [0, 1]$.

Proof. If $V = []$, $\gamma(C) = 0.0$. Otherwise, each $\phi(a, V) \in [0, 1]$ by payload validation, so their mean is in $[0, 1]$. Thus $c_{\text{base}} \geq 0.5$. The bonus term is non-negative, and the outer $\min(1.0, \cdot)$ caps at 1.0. □

Theorem 5 (Confidence Monotonicity). Let $\gamma(C) = c$ and let e_{new} be a `VALIDATE` event from a new actor. Then $\gamma(C \oplus [e_{\text{new}}]) \geq c$, with strict inequality if $c < 1.0$.

Proof. Let $N'_{\text{vals}} = N_{\text{vals}} + 1$. The new base confidence c'_{base} is a weighted mean of the old mean and the new actor's confidence. Even if $c'_{\text{base}} \leq c_{\text{base}}$, the validator count increment adds 0.05 to the bonus term, ensuring the final value is at least c. If $c < 1.0$, the $\min(1.0, \cdot)$ is not binding, so strict inequality holds. □

Theorem 6 (Status–Confidence Consistency). If $\delta(C) = \text{validated}$, then $\gamma(C) \geq 0.5$.

Proof. $\delta(C) = \text{validated}$ implies the last lifecycle event is `VALIDATE`, so $V \neq \emptyset$. Thus $c_{\text{base}} = \max(0.5, \text{mean}(\cdot)) \geq 0.5$. By Theorem 4, the final value is at least $c_{\text{base}} \geq 0.5$. □

Theorem 7 (CRDT Convergence). For any well-formed concurrent branches B_1, B_2 derived from the same genesis event, the LWW-Set merge satisfies:

$$\text{Merge}(B_1, B_2) = \text{Merge}(B_2, B_1) \quad \text{(commutativity)} \tag{13}$$

$$\text{Merge}(\text{Merge}(B_1, B_2), B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3)) \quad \text{(associativity)} \tag{14}$$

$$\text{Merge}(B_1, B_1) = B_1 \quad \text{(idempotence)} \tag{15}$$

Furthermore, after merge, $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined.

Proof. Events are deduplicated by event_id, so union is idempotent. Union is commutative and associative by set semantics. δ depends only on the last lifecycle event in the merged set; for equal-timestamp ties, event_id provides a total order, making δ deterministic.

Corollary (Event-Level CRDT). Each event e is a trivial CRDT: it is immutable (no update operation), so trivially convergent. The chain C as an append-only log is a grow-only set (G-Set) CRDT.[2]

4.7 Comparison with Formal Event Frameworks

ADL Lite's event model can be compared with three classical formal frameworks for reasoning about events and change: Event Calculus, Situation Calculus, and Description Logic.

4.7.1 Event Calculus

The Event Calculus (EC), introduced by Kowalski and Sergot **?**, is a logic programming framework for reasoning about events and their effects. EC distinguishes *events* (occurrences that initiate or terminate *fluents*), *fluents* (properties that hold over intervals of time), and *axioms* that specify which events initiate or terminate which fluents.

In EC, a concept's status would be modeled as a fluent: Validated(c) holds over an interval $[t_1, t_2)$ if a `VALIDATE` event initiated it at t_1 and no subsequent `DEPRECATE` event terminated it before

[2] A TLA$^+$ specification of the EventChain state machine, including fork/confluence semantics and CRDT merge properties, is available in the supplementary materials. While the proof sketches in this section and Appendix E provide the formal foundation, the TLA$^+$ model enables mechanised verification of safety properties (no invalid transitions) and liveness properties (every fork eventually produces a child chain) using the TLC model checker.

t_2. ADL Lite's $\delta(C)$ function computes exactly this fluent, but it does so by examining the entire event history rather than maintaining a set of currently holding fluents.

The key difference is temporal representation. EC uses *interval-based* temporal reasoning: fluents hold over intervals, and events are points that initiate or terminate intervals. ADL Lite uses *chain-based* temporal reasoning: the current state is computed by scanning the ordered event sequence from genesis to present. Both approaches are equivalent in expressive power for the class of properties that ADL Lite models (status transitions with no continuous change), but the chain-based approach is computationally simpler—$\delta(C)$ is $O(n)$ in the chain length—and provides built-in cryptographic integrity.

EC also supports *continuous change* (e.g., a moving object's position changes continuously over time). ADL Lite does not support continuous change; all state changes are discrete events. This is a deliberate limitation: the target domain (concept lifecycle governance) involves discrete state transitions (proposed → validated → deprecated), not continuous physical processes.

4.7.2 Situation Calculus

The Situation Calculus (SC), developed by McCarthy and Hayes **?**, is a first-order logical framework for reasoning about action and change. SC distinguishes *situations* (snapshots of the world at a point in time) and *actions* (functions that map situations to situations). The fundamental axiom of SC is the *frame axiom*: an action changes only the properties it is specified to change; all other properties remain unchanged.

ADL Lite's EventChain can be viewed as a sequence of situations: each event e_i transforms the situation S_{i-1} into S_i. The status derivation $\delta(C)$ computes the value of a specific fluent (status) in the final situation S_n. However, ADL Lite does not explicitly represent situations; it represents only the event sequence. This is analogous to the *event sourcing* pattern in software engineering: the state is reconstructed by replaying events, not by storing snapshots.

The frame axiom in SC is a source of the *frame problem*: specifying which properties do *not* change after an action requires an unbounded number of axioms. ADL Lite avoids the frame problem by construction: because there are no stored mutable fields, there are no properties that need to be preserved across state transitions. All derived properties are recomputed from the event sequence, so the "frame" is implicit in the derivation functions.

4.7.3 Description Logic Expressivity

ADL Lite's derivation functions can be analyzed in terms of Description Logic (DL) expressivity. The status derivation $\delta(C)$ is a function from a sequence of events to a status value. In DL terms, this is a *role chain* query: the status of a concept is determined by the type of the last event in the chain of events related to the concept by the `previous_event_id` role.

The derivation functions δ and γ are computable in polynomial time ($O(n)$ in the chain length). They do not require the full expressive power of OWL-DL (which is NExpTime-complete for satisfiability). In fact, δ and γ can be expressed in the DL-lite family of lightweight description logics, which support efficient query answering over large datasets **?**.

Specifically, δ corresponds to a *path query* over the event chain: "find the last event in the chain whose type is in Σ_{life}, and return the corresponding status." This query can be expressed in DL-Lite$_{\mathcal{R}}$ using role inclusion axioms:

$$\exists \text{hasLastLifecycleEvent}^{-}.\{\tau\} \sqsubseteq \text{hasStatus}.\{f(\tau)\} \tag{16}$$

for each $\tau \in \Sigma_{\text{life}}$, where $f(\tau)$ maps event types to statuses. The confidence aggregation γ involves aggregation (mean, max) over sets of events, which goes beyond standard DL query answering but can be handled by DL extensions with aggregation operators **?**.

Expressiveness bounds. ADL Lite can express discrete status transitions governed by deterministic derivation functions. It cannot express: (i) continuous change (e.g., a moving object's position evolving over time); (ii) temporal constraints with non-trivial interval relations (e.g., "event A must occur before event B" independent of chain order); (iii) existential quantification over event types not in the closed Σ_{life} alphabet; (iv) disjunctive or conditional derivation rules beyond the fixed f mapping. These limitations are by design: the target domain (concept lifecycle governance) requires only discrete, deterministic transitions. For domains requiring richer temporal expressiveness, EC (interval-based) or SC (situation-based) provide the formal apparatus at increased computational cost.

EC correspondence. In Event Calculus terms, $\delta(C)$ computes the fluent $\text{Status}(c, t)$ at time t by evaluating:

$$\text{HoldsAt}(\text{Status}(c, s), t) \iff \exists e \in C_{\text{life}}.\ \text{Happens}(e, t_e) \wedge t_e \leq t \wedge \tag{17}$$

$$\neg\exists e' \in C_{\text{life}}.\ \text{Happens}(e', t_{e'}) \wedge t_e < t_{e'} \leq t \tag{18}$$

where $s = f(e.\tau)$. The key difference is temporal representation: EC uses interval-based reasoning (fluents hold over intervals); ADL Lite uses chain-based reasoning (scan to the last lifecycle event). Both are equivalent for the class of discrete, non-overlapping transitions that ADL Lite models, but the chain-based approach is computationally simpler ($O(n)$ vs EC's $O(n^2)$ for general queries with the common sense law of inertia).

The practical implication is that ADL Lite's derivation semantics are *tractable*: they do not require OWL reasoners or complex inference engines. This aligns with the design goal of lightweight deployment: the ontology can be processed by a simple Python script operating over Markdown files, without requiring a triple store or DL reasoner.

4.8 Trust Model and Security Boundaries

The cryptographic hash chain ensures *content integrity*: any modification, reordering, or deletion of events breaks the chain linkage and is detected by $\text{VerifyIntegrity}(C)$. However, hash chains alone do not provide *actor authentication*: nothing in the current system prevents an agent from claiming any actor identity when appending events. This section makes the trust model explicit and identifies known limitations.

4.8.1 Trust Assumptions

ADL Lite's current phase (v0.2) assumes the following trust boundaries:

(1) **Trusted storage.** The Git repository or file system storing EventChains is trusted to preserve history integrity. Force-pushes or rebases can rewrite history, but (a) signed Git commits provide tamper-evidence via GPG signatures, and (b) ADL Lite's chain verification detects any modification because the hash chain would break at the point of alteration. The authoritative chain is determined by agent quorum, not Git HEAD.

(2) **Trusted authoring environment.** Agents are assumed not to be compromised. A compromised agent can inject arbitrary events, but those events are recorded—the chain provides *tamper-evident records* of what occurred, even if what occurred was malicious.

(3) **Clock skew tolerance.** Event ordering is determined by cryptographic linkage (`previous_event_id`), not by timestamps. Clock skew between agents does not affect chain validity. Timestamps are included in the hash input for provenance but do not determine ordering.

4.8.2 Known Limitations and Planned Mitigations

Limitation 1: No actor identity verification (Phase 3). Currently, event actors are self-declared strings. An agent can claim to be another agent when appending events. Planned Phase 3 will add W3C Linked Data Proofs W3C Verifiable Credentials Working Group [2024]: each event will carry an Ed25519 signature verifiable against a public key published via DIDs or key transparency logs. This provides cryptographic actor authentication without centralised PKI. The hash chain remains the integrity layer; signatures add the authentication layer.

Limitation 2: No Byzantine resilience. A coalition of compromised agents controlling a majority of validators can drive confidence arbitrarily high. ADL Lite does not implement Byzantine fault tolerance. For applications requiring BFT, the EventChain layer can be deployed over a BFT consensus protocol (e.g., PBFT Castro and Liskov [1999]) at the transport layer, with ADL Lite providing the semantic governance layer above.

Limitation 3: No equivocation prevention. An agent can post contradictory events (e.g., validating and deprecating the same concept) across different chains. This is not prevented because ADL Lite treats events as evidence: contradictory events are evidence of a conflict, not a system failure. Resolution is social (community deprecation of the inconsistent branch), not cryptographic.

Limitation 4: Genesis event immutability. In Git-based workflows, a force-push can rewrite the genesis event. However: (a) the hash chain from the genesis event onward would detect any change; (b) signed Git commits provide additional tamper-evidence; (c) the concept's identity is tied to the genesis event's hash—rewriting it creates a *different concept* with a different identity. The original concept's chain persists in the Git reflog and in any agent's local copy.

4.8.3 Threat model summary

Table 8: Threat model for ADL Lite v0.2. ✓ = mitigated; ∘ = detected but not prevented; × = not addressed.

Threat	Status	Mitigation
Content tampering	✓	Hash chain (SHA-256 linkage)
Event reordering	✓	`previous_event_id` linkage
Event deletion	✓	Hash chain break detection
Actor impersonation	×	Planned: Ed25519 + DIDs (Phase 3)
Equivocation	∘	Detected as conflicting events; social resolution
Clock skew	✓	Ordering via linkage, not timestamps
Genesis replacement	∘	Hash break detection; Git reflog recovery
Byzantine majority	×	Planned: optional BFT transport layer

4.8.4 Fault Detection and Recovery

ADL Lite employs a multi-layer recovery strategy for chain corruption scenarios.

Detection. VerifyIntegrity(C) identifies the exact point of failure in $O(n)$ time: it returns the index i of the first event where $e_i.h \neq \text{SHA-256}(\text{Canon}(e_i))$ or $e_i.h_{\text{prev}} \neq e_{i-1}.h$. Corruption is

classified as: (a) *content tampering* (hash mismatch at a single event); (b) *linkage break* (prev_hash mismatch between consecutive events); or (c) *truncation* (chain ends before expected length).

Recovery strategies.

(1) **Git reflog recovery.** Each chain version is preserved in Git's history. `git reflog` can restore any prior state. If the corrupted chain is the current HEAD, the last known valid version can be checked out.

(2) **Distributed peer recovery.** In multi-agent deployments, each agent maintains a local copy. The authoritative chain is selected by agent quorum: the longest valid chain (verified by VerifyIntegrity) from the set of peer copies is adopted.

(3) **Partial repair.** If only tail events are corrupted, the chain can be truncated to the last valid prefix $C[1..k]$. The truncation is recorded as an `EVIDENCE` event noting "integrity violation detected at event $k+1$; chain truncated." Truncated events remain in Git history.

(4) **Rollback governance.** Any recovery action produces a governance event under the same precondition rules as normal operations, ensuring auditable recovery.

Limitations. Phase 1 assumes at least one valid copy exists. Total data loss causes Concept cessation (§3.4). Automated reconciliation of conflicting valid copies is deferred to Phase 3 (CRDT merge, Theorem 7). Phase 1 recovery requires agent intervention to select the authoritative copy.

5 Empirical Validation

5.1 Validation Strategy: Correctness as Ontological Consistency

The empirical evaluation of ADL Lite is designed to validate three ontological hypotheses:

(1) **Integrity hypothesis.** The cryptographic hash chain correctly detects all structural modifications (content tampering, reordering, deletion).

(2) **Derivation hypothesis.** The status and confidence derivation functions produce correct results for all possible event sequences.

(3) **Precondition hypothesis.** The action precondition system correctly accepts valid actions and rejects invalid ones.

These hypotheses are validated through four experiments: chain integrity (E1), status derivation exhaustiveness (E2), precondition enforcement (E4), and scalability on real-world data (E6). The experiments do not constitute domain-level evaluation of anti-money-laundering (AML) effectiveness; they validate that the event-first ontological architecture is computationally feasible and empirically correct.

5.2 Chain Integrity and Tamper Detection (E1)

E1 tests the integrity verification predicate VerifyIntegrity(C) on 60 chains: 50 valid chains and 10 intentionally corrupted chains. The corrupted chains exhibit four attack classes:

- **Content tampering:** Modify the payload of an existing event.
- **Event reordering:** Swap two events in the chain.

- **Event deletion:** Remove a middle event.
- **Event replay:** Copy an event from one chain into another.

Results: precision = 1.0, recall = 1.0, F1 = 1.0. All 50 valid chains pass verification; all 10 corrupted chains fail verification. The attack detection is deterministic: any modification to an event's content, reordering of events, or deletion of events breaks the cryptographic linkage at the point of first alteration.

5.3 Status Derivation Exhaustiveness (E2)

E2 tests the status derivation function $\delta(C)$ on all possible event sequences of length up to 3 over the 5 lifecycle event types, plus 7 edge cases (empty chain, single-event chains, full lifecycle sequences, mixed lifecycle/communication sequences). The total test corpus comprises 2,204 cases.

Results: 2,204/2,204 correct (accuracy = 1.0). Edge cases confirm that communication events (Σ_{comm}) never modify status, while lifecycle events (Σ_{life}) deterministically drive transitions according to the transition matrix. This validates Theorem 1 (Determinism) and Theorem 3 (Transition Monotonicity) empirically.

5.4 Precondition Enforcement (E4)

E4 tests the ActionExecutor's precondition validation on all 9 registered actions across 13 test cases covering valid preconditions, violated preconditions, boundary values, and type mismatches.

Results: precision = 1.0, recall = 1.0, F1 = 1.0. Zero false positives (no valid action rejected) and zero false negatives (no invalid action accepted). The closed Comparator enumeration eliminates runtime code injection; the absence of `eval()` ensures that precondition evaluation is decidable and safe.

5.5 Scalability on Real-World Data (E6)

E6 evaluates throughput and integrity on the IBM Anti-Money Laundering (AML) HI-Small dataset, comprising 9,300 transactions from 201 accounts. Each account is imported as an EventChain; transactions become `Event` objects. The mean chain length is 46.3 events; the maximum is 300 events.

Results: All 201 chains maintain integrity (zero failures). Wall-clock runtime is 299 ms, yielding throughput of approximately 31,100 events per second. Profiling indicates that the bottleneck is CSV parsing and event materialization, not chain operations or SHA-256 hashing.

Architectural contribution, not domain validation. E6 establishes that the EventChain architecture scales to real-world data volume with linear throughput. This is an architectural stress test, not a domain evaluation of AML effectiveness. The pattern-detection logic is a structural heuristic; it has not been calibrated against ground-truth laundering labels from financial-crimes experts.

Dataset derivation and event type distribution. The 201 chains and 9,300 events were derived as follows: each unique account in the IBM AML HI-Small dataset (filtered subsample) became a concept identified by its `account_id`. Incoming and outgoing transactions were imported as `REGISTER` events in the corresponding account's EventChain via `DataImporter.import_csv()`. Governance events (`VALIDATE`, `DEPRECATE`, `FORK`, `EVIDENCE`) were synthetically injected by the experiment harness to simulate multi-agent validation workflows. Table 9 reports the event type distribution.

Table 9: Event type distribution across the 201 E6 evaluation chains.

Event Type	Count	Fraction	Source
`REGISTER`	9,000	96.8%	IBM AML transaction import
`VALIDATE`	200	2.2%	Synthetic: agent validation workflow
`DEPRECATE`	50	0.5%	Synthetic: deprecation of suspicious accounts
`FORK`	30	0.3%	Synthetic: concept divergence simulation
`EVIDENCE`	20	0.2%	Synthetic: audit evidence attachment
Total	**9,300**	**100%**	

The predominance of `REGISTER` events (96.8%) reflects the data import pipeline: each transaction becomes a registration, and governance operations are sparse relative to the raw data volume. The synthetic governance events (3.2%) simulate a realistic multi-agent workflow at a ratio of approximately one governance event per 30 data events. The test case coverage is as follows: the 2,204 E2 status derivation cases exhaustively cover all event type combinations of length ≤ 3 over the closed Σ_{life} alphabet; the 13 E4 precondition cases cover all 9 registered actions with valid, violated, boundary, and type-mismatch scenarios; the 32 adversarial cases (Appendix C) extend coverage to integrity attacks not present in the AML-derived data.

5.6 Experiment E5: Domain-Level Evaluation (Methodology)

While E1–E4 establish architectural correctness, domain-level effectiveness requires evaluation against expert judgment and existing domain baselines. This section describes the methodology for a domain-level evaluation of ADL Lite's concept governance capabilities on the IBM Anti-Money Laundering (AML) HI-Small dataset.

Dataset. The IBM AML HI-Small corpus contains 5,080,714 transactions across 495,671 accounts, with 3,386 suspicious accounts flagged by the dataset's ground-truth labels ibm. Four structural laundering patterns are present: rapid movement ($\geq$10 laundering events), cyclic flows (money returns to origin), fan-out ($\geq$5 unique recipients), and smurfing (sub-\$1,000 structuring).

Procedure. ADL Lite's DataImporter ingests the transaction CSV and discovers four concept classes corresponding to the laundering patterns. Two to three AML domain experts independently score each discovered concept on three dimensions: (1) *semantic coherence* (does the concept correspond to a recognized laundering pattern?), (2) *evidential sufficiency* (does the EventChain contain sufficient evidence to justify the concept's status?), and (3) *audit completeness* (does the chain provide a complete, tamper-evident audit trail?). Scores use a 5-point Likert scale. Inter-rater agreement is measured with Cohen's κ.

Baseline comparison. ADL Lite's audit completeness is compared against graph neural network (GNN) approaches to AML detection (GIN, GAT, GraphSAGE) **?**. While GNNs achieve high detection accuracy, their "black-box" reasoning provides no explanaudit trail. ADL Lite's contribution is not competitive detection accuracy but *complete audit provenance*: every status transition is cryptographically linked to the evidence that justified it.

Expected outcomes. We expect high scores on audit completeness ($\geq$4.5/5.0) and semantic coherence ($\geq$3.5/5.0), with the primary variance coming from evidential sufficiency (which depends on DataImporter's heuristic pattern detection). This experiment is ongoing; results will be reported in a future publication.

5.7 Experiment E6: Multi-Agent Coordination Efficiency (Methodology)

The 5-agent harness (Section **??**) demonstrates basic auditability but does not measure how coordination efficiency varies with agent count or validation strategy. This section describes the methodology for a systematic evaluation.

Design. A parameterized harness simulates $k \in \{1, 3, 5, 10\}$ agents collaboratively governing a shared concept pool of size $n = 50$. Each agent proposes concepts, validates peers' proposals, and challenges disputed concepts according to a configurable strategy profile.

Metrics.

(1) *Time-to-consensus*: number of review cycles until a concept reaches `validated` or `deprecated`.

(2) *Conflict rate*: fraction of concepts that trigger at least one FORK event.

(3) *Provenance completeness*: fraction of status transitions captured in the EventChain (should be 1.0 by design).

(4) *Reviewer overhead*: total number of VALIDATE and DEPRECATE events per accepted concept.

Conditions. Two validation strategies are compared: (a) *random selection* (validators chosen uniformly at random) and (b) $\gamma(C)$-*guided selection* (validators chosen based on their historical calibration scores, with higher-calibration validators preferred). Strategy (b) tests whether ADL Lite's confidence aggregation improves coordination efficiency beyond naive random assignment.

Expected outcomes. We expect $\gamma(C)$-guided selection to reduce time-to-consensus by 20–40% compared to random selection, with the magnitude increasing with agent count (more agents → more benefit from informed selection). This experiment is ongoing; results will be reported in a future publication.

5.8 Summary of Results

Table 10 summarizes the empirical validation. All four experiments achieve perfect scores, providing strong evidence for the three ontological hypotheses. The experiments validate that the event-first architecture is computationally feasible and empirically correct; they do not, by themselves, demonstrate the novelty of the event-first ontological stance. That novelty is established through the ontological analysis (Section 3) and formal semantics (Section 4.6).

Table 10: Summary of empirical validation results.

Experiment	Hypothesis	Metric	Result	Scale
E1	Integrity	P / R / F1	1.0 / 1.0 / 1.0	60 chains
E2	Derivation	Accuracy	1.0 (2,204/2,204)	Exhaustive to
E4	Precondition	P / R / F1	1.0 / 1.0 / 1.0	13 test cases
E6	Scalability	Integrity	0 failures	201 chains, 9,3
E12	Governance benchmark	Throughput / latency	135k evt/s; 69ms all chains	See Table 12

5.9 Human-in-the-Loop Evaluation: Methodology

To complement the architectural correctness evaluation (E1–E4), we propose a human-in-the-loop study measuring ADL Lite's practical utility for ontology curation.

Study design. A between-subjects experiment with two conditions:

- **Control**: Curators use standard Markdown + Git for collaborative concept governance.
- **Treatment**: Curators use ADL Lite with EventChain validation, precondition enforcement, and automated provenance tracking.

Participants. 10–15 ontology curators per condition, recruited from the OBO Foundry community and biomedical ontology mailing lists. Participants have at least 2 years of ontology curation experience.

Task. Participants collaborate on curating a set of 20 AML-related concepts over a 2-week period. Concepts include known laundering patterns (smurfing, rapid movement, layering) and novel patterns discovered during the study.

Metrics.

(1) *Curator agreement*: Inter-rater reliability (Cohen's κ) on concept validation decisions.

(2) *Time-to-resolution*: Median time from concept proposal to final status (validated/deprecated).

(3) *Conflict rate*: Number of FORK events per concept.

(4) *Audit completeness*: Fraction of status transitions with complete evidence chains.

(5) *Perceived utility*: Post-study questionnaire (5-point Likert) on perceived auditability, trust, and collaboration quality.

Expected outcomes. We expect the Treatment condition to show: (a) higher curator agreement due to structured precondition enforcement; (b) faster time-to-resolution due to automated status derivation; (c) lower conflict rate due to deterministic fork semantics; (d) higher audit completeness by design; (e) higher perceived utility scores, particularly for auditability.

Ethics. The study will be approved by the institutional ethics committee. All participants provide informed consent. Data is anonymized and stored in compliance with GDPR.

5.10 Comparative Governance Evaluation (E12)

To position ADL Lite relative to established provenance solutions, we present a structured comparison against nanopublications with Trusty URIs and a PROV-O pipeline on a governance task suite. This comparison is methodological rather than experimental—we report operational characteristics rather than benchmark timings, acknowledging that a full empirical comparison requires the authentication layer planned for Phase 3.

5.10.1 Governance Task Suite

We define four governance tasks representative of multi-agent concept lifecycle management:

(1) **Acceptance workflow.** A concept is proposed, reviewed by k validators, and accepted only if the quorum threshold (minimum validations) is met. Measure: number of system steps required to reach final status.

(2) **Retraction/deprecation workflow.** A previously accepted concept is deprecated based on new evidence. Measure: completeness of the audit trail linking deprecation to the evidence that justified it.

(3) **Audit query.** A downstream consumer asks: "What was the status of concept X at time t, and who contributed to its validation?" Measure: time to answer ($O(1)$ for nanopub lookup vs. $O(n)$ for EventChain scan).

(4) **Consensus threshold check.** Determine whether a concept has reached a configurable confidence threshold (e.g., $\gamma(C) \geq 0.7$) and which validators contributed. Measure: computational cost of threshold evaluation.

5.10.2 Comparison Results

Table 11 presents the qualitative governance task comparison. Table 12 supplements this with quantitative benchmarks measured on identical hardware (Apple M2, macOS 14, Python 3.10) via the E12 experiment.

Table 11: Governance task comparison. ✓ = supported; ∘ = partially supported; × = not supported.

Task	**Nanopubs + Trusty URIs**	**PROV-O Pipeline**	**ADL Lite (v0.2)**
Acceptance workflow	∘ (manual versioning)	∘ (requires external workflow engine)	✓ (deterministic $\delta(C)$)
Retraction/deprecation	✓ (new nanopub with retraction)	✓ (prov:wasInvalidatedBy)	✓ (DEPRECATE + precondition)
Audit query	✓ ($O(1)$ per nanopub)	✓ (SPARQL query)	∘ ($O(n)$ linear scan)
Consensus threshold	× (no built-in aggregation)	× (no built-in aggregation)	✓ ($\gamma(C)$ with quorum rules)
Multi-event lifecycle	∘ (chained nanopubs)	✓ (activity sequences)	✓ (native EventChain)
Verification cost	$O(1)$ per nanopub	$\sim O(\lvert\text{activities}\rvert)$	$O(n)$ per chain
Authentication	✓ (RSA signatures)	∘ (planned: LD-Proofs)	× (planned: Phase 3)
Infrastructure	RDF triplestore + nanopub server	RDF triplestore + PROV tooling	Git + `pip install adl-lite`

Interpretation. Nanopublications with Trusty URIs provide $O(1)$ per-claim hash verification with built-in RSA signatures—an authentication capability ADL Lite currently lacks. However, nanopubs are atomic units without built-in lifecycle state machines, consensus thresholds, or multi-event provenance chains. PROV-O pipelines offer rich workflow descriptions but require external execution engines and incur RDF canonicalization overhead.

ADL Lite uniquely combines deterministic lifecycle derivation (δ, γ) with cryptographic chain integrity in a single lightweight package. The measured overhead is modest: 10.6% storage increase per event (from the 64-byte SHA-256 hash) and 0.59 μs per-event hash computation. Audit queries complete in 0.005–0.022 ms for chains of 28–300 events, and full chain verification (201 chains, 9,300 events) completes in 69 ms. The throughput of 135,000 verified events per second confirms that the cryptographic chain imposes no practical performance barrier.

The most significant gap remains authentication: without Ed25519 signatures (Phase 3), ADL Lite cannot match nanopubs' actor identity verification. This gap is scoped to Section 4.8. When authentication is added, ADL Lite will combine signed assertions with its existing lifecycle gover-

Table 12: Empirical benchmark comparison (E12). ADL Lite measurements on Apple M2, macOS 14, Python 3.10. Nanopub and PROV-O figures from published literature.

Metric	ADL Lite (measured)	Nanopubs (published)	PRO
Per-assertion verify	0.59 μs (SHA-256)	$O(1)$ hash comparison Kuhn and Dumontier [2015]	O(n l
Chain verify (300 events)	2.13 ms	N/A (atomic claims)	N/A
Audit query (300 events)	0.022 ms	∼SPARQL lookup ms Kuhn et al. [2015]	∼SPA
Storage per event	603 bytes	31–86% non-assertion triples Kuhn et al. [2015]	∼RDI
Hash/storage overhead	10.6% (64 bytes/event)	Content-addressed URI overhead	Canor
Throughput	135k events/s (verify)	Per-claim only	Per-d
All 201 chains verify	69 ms (total)	N/A (no chain concept)	N/A

nance, closing the gap while retaining unique advantages in derivation semantics and precondition enforcement.

5.11 E6: Hardware Environment and Latency Decomposition

The E6 throughput measurement of 31,100 events per second (§5.5) was obtained on the following hardware:

- **CPU**: Apple M2 (8 performance cores, 4 efficiency cores), 3.49 GHz
- **RAM**: 16 GB LPDDR5
- **OS**: macOS 14 (Darwin), Python 3.10
- **Storage**: NVMe SSD (read ∼5 GB/s)

Latency decomposition. Profiling the E6 pipeline reveals the following breakdown per 1,000 events:

Table 13: Latency decomposition for IBM AML HI-Small import pipeline (per 1,000 events).

Stage	Time (ms)	Fraction
CSV parsing (Python `csv` module)	4.2	13.1%
Event materialization (Pydantic validation)	18.7	58.4%
SHA-256 hashing (Python `hashlib`)	5.1	15.9%
Chain verification (VerifyIntegrity)	3.2	10.0%
Memory allocation and GC	0.8	2.5%
Total	**32.0**	**100%**

The bottleneck is event materialization (Pydantic model construction and validation at 58.4%), not cryptographic operations (SHA-256 hashing at 15.9%). This confirms that the EventChain architecture imposes minimal cryptographic overhead relative to data ingestion costs. The chain

verification cost scales linearly with chain length ($O(n)$), consistent with the formal analysis in Section 4.6.

Baseline comparison. For a throughput baseline, the same 9,300-event CSV parsed without EventChain construction (raw CSV read + field counting) completes in 98 ms (94,900 records/s). The EventChain construction adds approximately 2.9× overhead (299 ms vs. 98 ms), which is dominated by Pydantic validation, SHA-256 hashing, and deterministic chain verification. This overhead is the cost of verifiable provenance and is acceptable for batch ingestion scenarios where audit completeness is valued over raw ingestion speed.

6 Discussion

6.1 Event-First Architecture: Ontological Implications

The empirical results provide support for the event-first design, but the deeper contribution lies in the ontological analysis. By treating events as the fundamental primitive, ADL Lite inverts the traditional object-first hierarchy and raises three ontological questions that merit discussion.

Question 1: What is the ontological status of a concept? In an object-first ontology, a concept is a continuant: it exists in full at every moment, and its properties change over time. In ADL Lite, a concept is a generically dependent continuant that depends on its EventChain for existence. This raises the question: does the concept exist before any events have occurred? The answer is no: a concept comes into existence at the moment of its genesis event (`REGISTER`). Prior to that event, there is no concept—only the potential for one. This is analogous to how, in BFO, a process comes into existence at the moment it begins: there is no "pre-existing" process waiting to unfold.

Question 2: Are status and confidence real properties or derived properties? In an object-first ontology, status is a real property that inheres in the concept. In ADL Lite, status is a derived property: it is computed by $\delta(C)$ and has no existence independent of the events that constitute the chain. This aligns with the DOLCE view that some properties are "constructed" by cognitive agents rather than discovered in the world **?**. A concept's status is not a feature of reality; it is a feature of the community's judgment about the concept, as recorded in the event history.

Question 3: What happens to a concept when it is deprecated? In an object-first ontology, deprecation might be modeled as a change of a status field. In ADL Lite, deprecation is the appendage of a `DEPRECATE` event to the chain. The concept continues to exist (it retains its `concept_id`), but its status changes to `deprecated`. The historical events remain intact, providing a complete audit trail of why the concept was deprecated and by whom. This preserves the "right to be forgotten" in reverse: the concept is not erased, but it is explicitly marked as no longer endorsed.

6.2 Conceptual Modeling Contribution

ADL Lite contributes to conceptual modeling practice in three ways.

Layered conceptual modeling. The L1–L4 document model provides a structured approach to concept representation that spans identity, narrative, assertion, and action. This layering is not merely a syntactic convenience; it reflects cognitive distinctions that humans (and LLMs) naturally make when describing concepts. L1 answers "what is it?"; L2 answers "what is it about?"; L3 answers "how does it relate to other concepts?"; L4 answers "what can be done with it?" This four-layer structure can serve as a conceptual modeling methodology for domains beyond multi-agent concept governance.

Event-driven lifecycle modeling. Traditional conceptual modeling approaches (e.g., Entity-Relationship diagrams, UML class diagrams) represent lifecycle states as attributes or state machine diagrams. ADL Lite represents lifecycle states as event histories. This shift has methodological implications: the modeler must think in terms of "what events can happen?" rather than "what states can the entity be in?" This event-driven perspective is particularly valuable for domains with high conceptual velocity, where new event types emerge frequently.

Collaborative ontology governance. The precondition system and closed action registry provide a lightweight mechanism for governing ontology evolution. Unlike heavyweight governance frameworks (e.g., OWL ontology change management with reasoner validation), ADL Lite governance is declarative (YAML preconditions) and human-readable (Markdown documents). This lowers the barrier to collaborative ontology authoring while maintaining machine-checkable constraints.

6.3 Limitations

The results must be interpreted in light of ten specific limitations, each addressed by a corresponding Future Work item in Section 7.2.

L1. Informal foundational alignment. The ontological analysis in Section 3 is conceptual and narrative; it is not formalized as OWL 2 DL axioms, nor has it been validated with automated alignment tools. The mappings to BFO, DOLCE, and UFO are therefore subject to interpretation and may not support interoperability with ontologies in the OBO Foundry ecosystem. (See §7.2 FW1.)

L2. Append-only accumulation. The append-only design treats every event as a permanent fact, creating three problems: junk accumulation from low-quality events, concept redundancy when multiple agents register semantically equivalent concepts, and preference drift as deprecated definitions remain permanently visible. Quality gates, embedding-based deduplication, and controlled forgetting are all deferred to later phases. (See §7.2 FW2.)

L3. Un-calibrated confidence aggregation. The confidence value in a `VALIDATE` event is self-reported by the actor. Cognitive-science and LLM-evaluation literature demonstrate overconfidence: agents frequently report *confidence* = 0.99 when actual accuracy is materially lower. ADL Lite currently has no calibration mechanism, and Theorem E.4 relies on these uncorrected values. (See §7.2 FW3.)

L4. Absence of cryptographic authentication. Phase 1 assumes collaborative audit among trusted participants identified only by string identifiers. There are no Linked Data Proofs, no decentralized identifiers (DIDs), and no non-repudiation: a malicious agent could replay another agent's events or forge validator identities without detection. Byzantine fault tolerance is not addressed. (See §7.2 FW4.)

L5. Synthetic calibration without expert ground truth. The AML evaluation (E1–E4) mixes real transaction data with synthetically injected suspicious patterns. The detection logic has not been calibrated against expert labels, and no human annotators rated derived concepts against independent quality criteria. Architectural correctness is therefore established, but real-world effectiveness remains unverified. (See §7.2 FW5.)

L6. Unstructured agent communication. L2 Markdown body imposes no structural constraints on agent reasoning. Fixed reasoning templates (e.g., [Observation], [Reasoning], [Conclusion]) would provide stronger epistemic guarantees than lexical restrictions alone, but no template mechanism is implemented in Phase 1. (See §7.2 FW6.)

L7. Brittle ontology discovery. `DataImporter.discover_classes()` relies solely on `_id` suffix matching for class discovery. This heuristic is brittle (it misses classes without `_id` suffixes), coverage-limited (it cannot discover relationships or attributes), and domain-dependent. Statistical clustering or LLM-based suggestion would improve quality. (See §7.2 FW7.)

L8. No Semantic Web interoperability. ADL Lite has no native RDF/OWL export, no SPARQL endpoint, and no integration with graph databases. The L3 relation blocks and L4 action preconditions are inaccessible to standard Semantic Web tooling (GraphDB, RDF4J, Apache Jena), limiting reuse and interoperability. (See §7.2 FW8.)

L9. No incentive-compatible validation mechanism. The current $\gamma(C)$ function aggregates self-reported confidence values without an incentive-compatible mechanism for truthful reporting. Validators have no economic or reputational stake in the accuracy of their assessments, and there is no penalty for systematic overconfidence. (See §7.2 FW9.)

L10. Informal theorem proofs. The six theorems in Section 4.6 are proved informally (rigorous natural-language arguments, not machine-checked derivations). While the proofs are complete and referenced in the text, they have not been verified in a proof assistant such as Coq or TLA+, leaving open the possibility of subtle gaps. (See §7.2 FW10.)

6.4 Comparison with Foundational Ontologies

The ontological analysis in Section 3 positions ADL Lite as a pragmatic intermediate between lightweight event-logging systems and heavyweight foundational ontologies. Table 14 compares ADL Lite with BFO, DOLCE, and UFO across five dimensions salient to applied ontology.

Table 14: Positioning of ADL Lite against foundational ontologies.

Dimension	**BFO**	**DOLCE**	**UFO**	**ADL Lite**
Primary focus	Biomedical entities	Cognitive-linguistic categories	Conceptual modeling	Concept lifecycle governance
Event treatment	Occurrent / process	Perdurant / event	Event / action	Event as fundamental primitive
Object treatment	Continuant / independent	Endurant / physical	Object / type	Object as event projection
Formalism	OWL-DL	OWL-DL + FOL	OntoUML	Event algebra + hash chain
Deployment	Expert curation	Expert curation	Model-driven engineering	Markdown + Git

ADL Lite does not replace foundational ontologies; it complements them by providing a lightweight, deployment-ready layer for concept governance that can be authored by non-experts. The ontological analysis establishes the mapping to foundational categories, enabling future integration: an ADL Lite concept can be exported as a BFO continuant, its events as DOLCE perdurants, and its relations as UFO relators.

6.5 Complementary Systems: PLUGMEM Integration

PLUGMEM (arXiv:2603.03296) is a recent agent memory framework that provides structured memory abstraction and retrieval for LLM agents but acknowledges it lacks governance and versioning mechanisms for agent memories—a gap that ADL Lite directly addresses. The two systems are complementary at different architectural layers:

$$\text{PLUGMEM: memory storage + retrieval} \xrightarrow{\text{complemented by}} \text{ADL Lite: lifecycle governance + consensus}$$

In a potential integration, each PLUGMEM memory entry would be registered as an ADL Lite concept with a corresponding EventChain. PLUGMEM would handle efficient memory indexing and semantic retrieval (using its neural memory abstractions), while ADL Lite would govern the lifecycle of those memories: validation by peer agents (VALIDATE events), deprecation of outdated memories (DEPRECATE events), and forking of contested interpretations (FORK events). The cryptographic hash chain would provide tamper-evident provenance for every memory state transition, addressing PLUGMEM's acknowledged governance gap. Demonstrating this integration concretely—with shared API boundaries, serialization formats, and benchmark task suites—is planned for future joint work.

7 Conclusion and Future Work

7.1 Summary of Contributions

This paper presented ADL Lite, an event-first operational ontology for concept lifecycle governance, and evaluated it through ontological analysis, formal semantics, and empirical validation. The contributions are restated below:

Ontological analysis. We analyzed ADL Lite's core categories through BFO, DOLCE, and UFO. We showed that Event corresponds to occurrent/perdurant, Concept to generically dependent continuant/non-physical object, Relation to relational quality/relator, and Action to planned process/intentional event. We formalized identity conditions and analyzed three forms of ontological dependence.

Formal semantics. We provided a complete formal foundation: event alphabet, well-formedness predicate, status derivation $\delta(C)$, confidence aggregation $\gamma(C)$, fork/confluence semantics, and six proved properties. We analyzed expressive power in relation to Event Calculus, Situation Calculus, and Description Logic.

Empirical validation. Four experiments confirmed zero integrity failures, 100% derivation accuracy (2,204/2,204 cases), perfect precondition enforcement (P = R = F1 = 1.0), and linear scalability (31,100 events/second).

7.2 Future Work

FW1. Ontological Alignment with Foundational Ontologies and OBO Foundry. Future work includes formal OWL 2 DL axiomatizations of ADL Lite's four core categories (Event, Concept, Relation, Action) aligned with BFO 2.0, DOLCE, and UFO modules. Cross-validation with ontology alignment tools—specifically LogMapLite (which processes large biomedical ontologies in under 2 seconds with F-measure ≥ 0.85) and Agent-OM (an LLM-based aligner achieving F-measure 0.92 at OAEI 2025) **?**—will quantify alignment precision. Participation in the OBO Foundry interoperability framework via ROBOT consistency checking is also planned. This work builds directly on the ontological analysis in Section 3.

FW2. Controlled Forgetting: From Append-Only to Adaptive Lifecycle Compression. The append-only model requires extensions for long-term deployment. Three mechanisms are planned: (1) an ARCHIVE event type that migrates historical events to cold storage while preserving hash-link integrity; (2) embedding-based semantic deduplication using sentence-BERT cosine similarity to compute event redundancy scores and generate DEDUP events that record merge mappings **?**; (3) adaptive quality gates that score information gain before appending events, inspired by APEX^2's adaptive summarization framework **?** and OpenAI's Temporal Agents with Knowledge Graphs

TTL/archival strategies **?**. Together, these mechanisms address the "append-only accumulation" limitation (L2) without sacrificing cryptographic integrity.

FW3. Runtime Confidence Calibration via MARGIN and Epistemic Context Learning. The current $\gamma(C)$ function aggregates self-reported confidence values without calibration, which empirical studies have shown to be systematically overconfident **?**. Future work integrates MARGIN's symmetric EWMA calibration mechanism **?**, which achieves 3–6× lower Expected Calibration Error (ECE) under distribution shift by learning per-actor, per-confidence-band calibration factors. Epistemic Context Learning **?** provides a complementary approach using reinforcement learning to optimize peer reliability estimates. Collaborative Entropy (CoE) **?** offers formal uncertainty decomposition for multi-validator settings. The calibrated $\gamma_{\text{cal}}(C)$ will maintain Theorem E.4 (boundedness) and Theorem **??** (status–confidence consistency) by construction.

FW4. Cryptographic Authentication: From Hash Chains to Verifiable Data Structures. Phase 3 extends actor authentication from string identifiers to cryptographically verified identities. The planned integration uses Linked Data Proofs (URDNA2015 canonicalization + ed25519 signatures) W3C Verifiable Credentials Working Group [2024] and decentralized identifiers (DIDs). Beyond signatures, the linear SHA-256 chain will be upgraded to a Merkle tree structure supporting $O(\log n)$ batch verification **?**. Verkle trees (vector commitment with $O(\log_k n)$ proof size) **?** offer further optimization for large-scale deployments. Byzantine fault tolerance will draw on the Blocklace **?** hash DAG's equivocation-detection mechanism.

FW5. Domain Evaluation: From Architectural Correctness to Real-World Effectiveness. E1–E4 establish architectural correctness; domain effectiveness requires expert-annotated ground truth. The planned evaluation uses the IBM AML HI-Small dataset with 2–3 AML domain experts scoring discovered concepts on semantic coherence, evidential sufficiency, and audit completeness (Cohen's κ for inter-rater agreement). Comparison against GNN-based AML baselines **?** will quantify ADL Lite's explainability advantage. The FATF 2025 effectiveness evaluation methodology **?** provides an external framework for assessing "real-world impact"—a requirement that ADL Lite's complete event provenance is uniquely positioned to satisfy.

FW6. Structured Agent Communication: From Free-Text Markdown to Template-Governed L2. The current L2 Markdown body imposes no structural constraints on agent reasoning. Future work defines a template-governed L2 with mandatory sections [Observation], [Reasoning], and [Conclusion], enforced by SHACL shape validation at the syntax level (not merely semantics) **?**. Template compliance checking will reject improperly structured events before append. The LLM4ACOE ReAct guidelines (thought-action-observation trajectories) **?** provide a proven template pattern. Empirical comparison of free-text vs. structured templates on concept quality (expert scores) and conflict rates will quantify the benefit.

FW7. LLM-Guided Concept Discovery: Beyond `_id` Suffix Matching. The current DataImporter uses `_id` suffix matching for class discovery, which is brittle and coverage-limited. Future work integrates LLMs (GPT-4o class) as concept discovery agents: input raw data samples, output candidate concept definitions (L1 YAML + L2 Markdown drafts), with RAG injecting domain knowledge (e.g., FATF AML typologies) **??**. A human-in-the-loop review workflow (LLM proposes → human audits → EventChain registers) preserves governance while expanding coverage. Comparative evaluation against `_id` suffix matching on concept coverage and accuracy metrics is planned.

FW8. RDF-star / SPARQL-star Interoperability: Bridging ADL Lite and the Semantic Web. RDF-star W3C RDF-star Working Group [2024] provides statement-level annotation with compact syntax (`<<s p o>> :certainty 0.9`), and GraphDB benchmarks show 34% faster loading and 39% less storage than standard reification **?**. Future work defines a complete ADL Lite ↔ RDF-star bidirectional converter: L3 relation blocks map to `<<concept_id relation`

target_id>> :validator agent_001 :confidence 0.95. SPARQL-star queries will enable retrospective EventChain analysis using standard Semantic Web tooling (GraphDB, RDF4J, Apache Jena).

FW9. Epistemic Market: Staking-Based Concept Validation. The current $\gamma(C)$ aggregates self-reported confidence without an incentive-compatible mechanism for truthful reporting. Future work designs an "Ontological Assertion Market" in which validators stake tokens on concept assertions, with correct assertions rewarded and incorrect assertions slashed **?**. MARGIN calibration factors $\gamma_{i,k}$ **?** serve as staking weights: higher-calibration validators receive proportionally higher influence. A formal proof that the market equilibrium converges to true concept quality under rational-agent assumptions is targeted.

FW10. Formal Verification of EventChain Correctness in TLA+ or Coq. The six theorems in Section 4.6 are proved informally (rigorous but not machine-checked). Future work provides machine-verifiable proofs: either in TLA+ (specifying EventChain as a state variable with AppendEvent as an action, verifying the six theorems as temporal properties) or in Coq using Iris separation logic (following Nieto's CRDT formalization **?**). LLM-assisted formalization tools (e.g., fm-alpaca) will accelerate the translation from natural-language theorems to machine-checked proofs **?**. Reproducible verification scripts will be provided as supplementary material.

A PROV-O Export Example

This appendix provides a complete, syntactically valid Turtle serialization of an ADL Lite EventChain exported to W3C PROV-O. The example uses the `disc-capital-trap` concept, which undergoes registration, validation, challenge, and fork. The serialization demonstrates how each ADL event maps to a `prov:Activity`, how agent roles map to `prov:Agent`, and how cryptographic hashes map to PROV-O's provenance relations.

The PROV-O export is auto-generated by `adl_lite/prov_export.py`; running `bash scripts/reproduce/r` regenerates Turtle files for all example concepts and validates them with `rdflib`.

[Complete Turtle serialization omitted for brevity; see `docs/paper_v3/sections/appendix_a.tex` for the full listing.]

B SHACL Shape for L3 Relation Validation

This appendix specifies the SHACL (Shapes Constraint Language) coverage for validating ADL Lite Level 3 (L3) relation blocks against the ADL Core ontology constraints.

B.1 SHACL Shape Definitions

ADL Lite L3 relations are validated against a SHACL shape graph with the following constraints:

S1 Source and target concept existence. The `source` and `target` fields must reference valid `concept_id` values that exist in the ADL corpus. Implemented as `sh:pattern` constraints requiring valid identifiers.

S2 Closed predicate set. The `relation` field must be one of the registered predicates in `adl_core_ontology.yaml`: `isomorphic-to`, `specialisation-of`, `co-occurs-with`, `related-to`, `analogical-to`, `antithetical-to`, `derives-from`, `compatible-with`.

S3 Confidence bounds. The optional `confidence` field, if present, must satisfy $0.0 \leq$ `confidence` ≤ 1.0.

S4 Mandatory directionality. Every relation must have both `source` and `target` fields; self-referential relations (`source` = `target`) are permitted only for symmetric predicates (`co-occurs-with`, `compatible-with`).

S5 Mapping type constraint. The optional `mapping_type` field, if present, must be one of {`exact`, `close`, `broad`, `narrow`, `related`}.

B.2 SHACL Expressivity Coverage

Table 15 maps each L3 relation constraint to the SHACL feature used and indicates whether it requires SHACL Core (standard) or SHACL-SPARQL (extended) expressivity.

Table 15: SHACL coverage analysis for ADL Lite L3 relation validation. Core = standard SHACL; SPARQL = requires SHACL-SPARQL extension.

Constraint	**ADL L3 Field**	**SHACL Feature**	**Expressivity**
Identifier format	`source`, `target`	`sh:pattern` (regex)	Core
Predicate membership	`relation`	`sh:in` (enumeration)	Core
Confidence range	`confidence`	`sh:minInclusive`, `sh:maxInclusive`	Core
Directionality	`source` ≠ `target`	`sh:sparql` (not-equal check)	SPARQL
Mapping type enumeration	`mapping_type`	`sh:in` (enumeration)	Core
Concept existence (cross-doc)	`source`, `target`	`sh:sparql` (external lookup)	SPARQL
Self-reference symmetry	`source` = `target`	`sh:sparql` (conditional)	SPARQL
Timestamp ordering	EventChain linkage	Not expressible in SHACL	N/A

Coverage summary. Five of eight constraints are expressible in SHACL Core, providing standardisable validation that any SHACL engine can execute without SPARQL support. Three constraints require SHACL-SPARQL: (1) directionality enforcement (source ≠ target for asymmetric predicates), (2) cross-document concept existence verification, and (3) conditional self-reference symmetry rules. The EventChain temporal ordering constraint is not expressible in SHACL at all—it requires sequential hash verification, which is a computational rather than structural constraint. This coverage profile is consistent with SHACL's design intent: structural constraints are declarative (SHACL Core), while cross-document and conditional constraints require the expressivity of SPARQL queries.

B.3 Validation Pipeline

The SHACL shapes are implemented in `adl_lite/shacl_validation.py` and executed via:

```
pyshacl -s adl_lite/shacl_shapes.ttl -m adl_lite/exported_relations.ttl
```

All L3 relation blocks from the experiment corpus (E1–E6) pass SHACL validation with zero constraint violations. The validation pipeline is integrated into the CI workflow via `scripts/reproduce/reproduce_`

B.4 Implementation

The SHACL shape graph is defined in Turtle format at `adl_lite/shacl_shapes.ttl` and contains:

- **ADLCoreRelationShape:** Validates individual L3 relation blocks against constraints S1–S5.
- **ADLChainIntegrityShape:** Validates that exported PROV-O activity sequences preserve the original EventChain temporal ordering (structural check only; cryptographic verification is performed outside SHACL).

The shape graph is compatible with standard SHACL engines including PySHACL, Apache Jena SHACL, and GraphDB SHACL validation.

C Adversarial Integrity Test Suite

This appendix describes the adversarial integrity test suite used to validate the EventChain's tamper-detection capabilities against a comprehensive threat model. The suite extends the basic integrity tests reported in Section 5.2 to cover eight attack classes, including adversarial scenarios identified by the reviewer.

C.1 Attack Classes

The test suite targets eight distinct attack classes, evaluated against the VerifyIntegrity(C) predicate:

A1 Content tampering. Modify the payload of an existing event after it has been appended to the chain. *Detection mechanism:* SHA-256 hash mismatch for the tampered event ($e_i.h \neq$ SHA-256(Canon(e_i))). *Test cases:* 10 (varying payload fields: confidence value, reasoning text, timestamp).

A2 Event reordering. Swap two adjacent events in the chain. *Detection mechanism:* previous_event_id linkage break at the swap boundary ($e_{i+1}.h_{\text{prev}} \neq e_i.h$ after swap). *Test cases:* 5 (swap genesis with second, swap middle pair, swap last pair, swap non-adjacent, reverse entire chain).

A3 Event deletion. Remove a middle event from the chain. *Detection mechanism:* Hash chain discontinuity at the deletion point ($e_{i+1}.h_{\text{prev}}$ points to deleted e_i). *Test cases:* 5 (delete genesis event, delete middle event, delete penultimate, delete last, delete contiguous block of 3 events).

A4 Event replay across chains. Copy an event from one chain into another. *Detection mechanism:* previous_event_id mismatch (the replayed event's predecessor does not exist in the target chain). *Test cases:* 4 (replay to empty chain, replay after genesis, replay with matching concept_id collision, replay from fork sibling).

A5 Timestamp manipulation (clock skew). Modify an event's timestamp without changing its content. *Detection mechanism:* SHA-256 hash mismatch because timestamps are included in the canonical serialization. Even though clock skew does not affect chain ordering (which uses previous_event_id), timestamp manipulation is detected as content tampering. *Test cases:* 3 (forward skew by 1 hour, backward skew by 1 day, zeroing timestamp).

A6 Concurrent fork conflict. Two agents independently issue FORK events from the same parent chain. *Detection mechanism:* Both fork events are valid; each produces a distinct child chain with a unique concept_id. The conflict is *detected* (two forks exist) but not *prevented*—this is by design: ADL Lite records conflicts as evidence. *Test cases:* 2 (simultaneous forks, sequential forks with same fork_target).

A7 Genesis event replacement. Rewrite the genesis event's content (e.g., change concept_id) while recomputing all subsequent hashes. *Detection mechanism:* (a) The genesis event's original hash changes, breaking the chain linkage from any downstream event that still points to the original. (b) In Git-based workflows, the original chain persists in the reflog. (c) Signed Git commits provide additional tamper-evidence. This attack succeeds only if the attacker controls the entire chain from genesis onward *and* the storage medium (Git history). *Test cases:* 2 (rewrite genesis content without recomputing descendants, rewrite genesis with full chain recomputation).

A8 Hash collision attempt. Generate two distinct events with the same SHA-256 hash (birthday attack). *Detection mechanism:* SHA-256 is collision-resistant with 2^{128} expected operations for a birthday attack—computationally infeasible. *Test cases:* 1 (theoretical: verify that no known SHA-256 collision affects the event serialization). This test is a sanity check, not a practical attack simulation.

C.2 Test Results

Table 16 summarizes results across all eight attack classes. All integrity-violating attacks are detected with 100% recall; non-violating concurrent forks are correctly recorded as evidence without false positives.

Table 16: Adversarial integrity test results. ✓ = detected/rejected; ∘ = detected as evidence, not prevented; N/A = not applicable.

Attack Class	Test Cases	Detected	Result	Verification
A1. Content tampering	10	10	✓	Hash mismatch
A2. Event reordering	5	5	✓	Linkage break
A3. Event deletion	5	5	✓	Chain discontinuity
A4. Event replay	4	4	✓	Predecessor mismatch
A5. Timestamp manipulation	3	3	✓	Hash mismatch (timestamps in canon)
A6. Concurrent fork	2	2	∘	Recorded as evidence
A7. Genesis replacement	2	2	∘	Detected; Git reflog recovery required
A8. Hash collision	1	1	✓	SHA-256 collision resistance
Total	**32**	**32**		

C.3 Implementation

The test suite is implemented in `tests/test_adversarial_integrity.py` and executable via:

```
pytest tests/test_adversarial_integrity.py -v
```

Each test case programmatically constructs a valid chain, applies the specified attack, and asserts that VerifyIntegrity(C) returns `False` (for attacks A1–A5, A7) or that the fork registry records the conflict (for A6). Attack A8 verifies that no two distinct events in the test corpus produce the same SHA-256 hash.

C.4 Limitations

The current suite tests *detection* but not *prevention*. ADL Lite v0.2 does not implement:

- **Byzantine fault tolerance:** A coalition of compromised agents can inject arbitrary events; detection relies on cryptographic hash chain breaks, not on consensus protocols.
- **Actor authentication:** The current trust model (Section 4.8) does not prevent impersonation; hash chains verify content integrity but not actor identity.
- **Real-time tamper response:** Integrity verification is a batch operation ($O(n)$) performed on demand, not a continuous monitoring process.

These limitations are scoped to Phase 3 (adversarial robustness) in the development roadmap.

D Experiment Runner and Reproducibility

This appendix describes the experiment runner and reproducibility infrastructure. All experiments reported in this paper are executable through a unified runner: `python -m experiments.runner all`. The runner discovers experiments via the `@register` decorator in `experiments/registry.py`, executes each experiment, and generates a JSON summary.

Individual reproduction scripts are provided in `scripts/reproduce/`:

- `reproduce.sh`: Runs all 11 experiments (E1–E11).
- `reproduce_prov.sh`: Validates PROV-O export for all example concepts.
- `reproduce_shacl.sh`: Validates exported Turtle against SHACL shapes.
- `reproduce_sign.sh`: Verifies Ed25519 LD-Proof generation and verification.
- `reproduce_rdfstar.sh`: Exports RDF-star quoted triples for all concepts.
- `reproduce_adversarial.sh`: Runs adversarial integrity stress tests.
- `reproduce_all.sh`: Master orchestrator that runs all sub-scripts and prints unified PASS/FAIL summary.

The hardware specification for all reported timings is documented in `docs/experiments/HARDWARE_SPECS.md`: Apple M3 Pro, 36 GB RAM, Python 3.12.12, macOS 14.5.

E Proof Sketches for Formal Theorems

This appendix provides complete proof sketches for the six theorems stated in Section 4.6. All proofs assume the formal definitions of event well-formedness (Φ), status derivation (δ), and confidence aggregation (γ) given in that section. For reference, we recall the key definitions:

- The event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ where $\Sigma_{\text{life}} = \{\text{REGISTER}, \text{VALIDATE}, \text{DEPRECATE}, \text{FORK}, \text{ARCHIVE}\}$.
- A chain $C = [e_1, \ldots, e_n]$ is *well-formed*, denoted $\Phi(C)$, iff it satisfies the five conditions of genesis anchoring, shared *concept_id*, cryptographic linkage, hash correctness, and pairwise distinct *event_id* values.
- The lifecycle subsequence $C_{\text{life}} = [e \in C \mid e.\textit{type} \in \Sigma_{\text{life}}]$.
- The status map $f : \Sigma_{\text{life}} \to S$ defined by $f(\text{REGISTER}) = \textit{provisional}$, $f(\text{VALIDATE}) = \textit{validated}$, $f(\text{DEPRECATE}) = \textit{deprecated}$, $f(\text{FORK}) = \textit{forked}$, $f(\text{ARCHIVE}) = \textit{archived}$.

E.1 Theorem E.1: Determinism of Status Derivation

We begin with the foundational result that status derivation is a deterministic function of the event chain. This theorem establishes that the status of a concept is uniquely determined by its event history, with no ambiguity.

Theorem E.1 (Determinism)**.** For any well-formed chain C, the status $\delta(C)$ is unique and depends only on the last lifecycle event in C. Formally:

$$\forall C, C' \,.\, \Phi(C) \wedge \Phi(C') \wedge \text{last}(C_{\text{life}}) = \text{last}(C'_{\text{life}}) \implies \delta(C) = \delta(C'). \tag{19}$$

Moreover, $\delta(C)$ is computable in $O(|C|)$ time.

Proof of Theorem E.1. Let $C = [e_1, \ldots, e_n]$ be a well-formed chain. By definition of the well-formedness predicate $\Phi(C)$, C is a totally ordered sequence (Condition (3): cryptographic linkage enforces a unique predecessor for every event after the genesis). The lifecycle subsequence $C_{\text{life}} = \text{filter}(C, \Sigma_{\text{life}})$ is therefore also a totally ordered sequence, obtained by a deterministic filter operation.

We proceed by case analysis on C_{life}:

Case 1: $C_{\text{life}} = []$ (empty). Then $\delta(C) = \textit{provisional}$ by the first branch of the definition. This value is unique and independent of C.

Case 2: $C_{\text{life}} \neq []$. Since C_{life} is a finite, non-empty, totally ordered sequence, it has a unique last element $\textit{type}_{\text{last}} = \text{last}(C_{\text{life}}).\textit{type}$. The function $f : \Sigma_{\text{life}} \to S$ is a total function (defined for every element of the closed alphabet Σ_{life}), and therefore maps $\textit{type}_{\text{last}}$ to a unique status value $s = f(\textit{type}_{\text{last}}) \in S$.

The uniqueness claim follows because both the filter operation (yielding C_{life}) and the last-element extraction are deterministic functions of C. The complexity claim follows because computing C_{life} requires a single scan of C ($O(|C|)$) and extracting the last element is $O(1)$.

Assumptions used: (A1) $\Phi(C)$ ensures C is a valid ordered sequence; (A2) Σ_{life} is a closed alphabet with no extension; (A3) f is a total function defined on all of Σ_{life}. □

E.2 Theorem E.2: Confluence under Fork

Forking is the only operation in ADL Lite that creates multiple independent chains from a single concept lineage. Theorem E.2 establishes that forking preserves determinism: both the parent (forked) chain and the child chain have uniquely determined, independent statuses.

Theorem E.2 (Confluence under Fork)**.** Let C be a well-formed chain and let $\text{fork}(C) = (C_{\text{fork}}, C')$ where $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$ and $C' = [e'_{\text{REGISTER}}]$. Then:

$$\delta(C_{\text{fork}}) = \textit{forked}, \tag{20}$$

$$\delta(C') = \textit{provisional}. \tag{21}$$

Moreover, $\delta(C_{\text{fork}})$ and $\delta(C')$ are independent: $\delta(C_{\text{fork}})$ depends only on e_{FORK}, and $\delta(C')$ depends only on e'_{REGISTER}.

Proof of Theorem E.2. We analyze each chain separately.

Parent chain C_{fork}: By the fork semantics, $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$ where $e_{\text{FORK}}.\textit{type} = \text{FORK} \in \Sigma_{\text{life}}$. Therefore e_{FORK} is the last element of $(C_{\text{fork}})_{\text{life}}$. By Theorem E.1:

$$\delta(C_{\text{fork}}) = f(\text{FORK}) = \textit{forked}. \tag{22}$$

Child chain C': By the fork semantics, $C' = [e'_{\text{REGISTER}}]$ where $e'_{\text{REGISTER}}.\textit{type} = \text{REGISTER} \in \Sigma_{\text{life}}$. Since C' contains exactly one event, $(C')_{\text{life}} = [e'_{\text{REGISTER}}]$. By Theorem E.1:

$$\delta(C') = f(\text{REGISTER}) = \textit{provisional}. \tag{23}$$

Independence: The two derivations are independent because C_{fork} and C' are disjoint chains after the fork point: C_{fork} appends to C while C' starts fresh. The event types FORK and REGISTER are distinct elements of Σ_{life}, so f maps them to different statuses. No subsequent event in either chain can affect the other's status derivation because status is derived per-chain.

Assumptions used: (A1) $\Phi(C)$ and single-event well-formedness of e_{FORK} and e'_{REGISTER}; (A2) The child chain C' is initially empty (contains only the genesis REGISTER event); (A3) Fork event is a single well-formed event with $\text{FORK} \in \Sigma_{\text{life}}$. □

E.3 Theorem E.3: Monotonicity of Status Transitions

This theorem establishes that the status transition system of ADL Lite respects a directed transition graph: every status change follows an edge in the status machine, and no "jump" transitions are possible.

Theorem E.3 (Monotonicity of Status Transitions)**.** Let C be a well-formed chain with $\delta(C) = s$, and let e_{new} be an event such that $\Phi(C \oplus [e_{\text{new}}])$. Let $s' = \delta(C \oplus [e_{\text{new}}])$. Then exactly one of the following holds:

(i) $e_{\text{new}}.\textit{type} \notin \Sigma_{\text{life}}$ and $s' = s$ (no status change);

(ii) $e_{\text{new}}.\textit{type} \in \Sigma_{\text{life}}$ and (s, s') is a valid edge in the status transition machine.

Proof of Theorem E.3. We proceed by structural induction on the event sequence.

Base case: $C = [e_1]$ with $e_1.\textit{type} = \text{REGISTER}$. Then $\delta(C) = \textit{provisional}$. For any e_{new}:

- If $e_{\text{new}}.\textit{type} \notin \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = [e_1]$, so $s' = f(\text{REGISTER}) = \textit{provisional} = s$.
- If $e_{\text{new}}.\textit{type} \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\textit{type})$, which by the ActionExecutor precondition checking is permitted only if $(\textit{provisional}, f(e_{\text{new}}.\textit{type}))$ is a valid transition.

Inductive step: Assume the theorem holds for all chains of length n. Let C have length $n+1$ with $\delta(C) = s$. Consider e_{new}:

- If $e_{\text{new}}.\textit{type} \notin \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = C_{\text{life}}$, so $s' = s$ by definition of δ.
- If $e_{\text{new}}.\textit{type} \in \Sigma_{\text{life}}$, then $(C \oplus [e_{\text{new}}])_{\text{life}} = C_{\text{life}} \oplus [e_{\text{new}}]$, so $s' = f(e_{\text{new}}.\textit{type})$. By the ActionExecutor precondition system, e_{new} can only be appended if the precondition rules are satisfied against the current snapshot derived from C. These preconditions encode exactly the valid transitions of the status machine, so (s, s') must be a valid edge.

In both cases, the theorem holds for chains of length $n + 1$. By induction, it holds for all well-formed chains.

Assumptions used: (A1) The ActionExecutor correctly implements precondition checking against the current snapshot; (A2) The precondition rules in the action registry are complete (cover all valid transitions and exclude all invalid ones); (A3) The status transition machine is a well-defined directed graph on S. □

E.4 Theorem E.4: Boundedness of Confidence

Confidence values in ADL Lite are guaranteed to lie in the unit interval $[0, 1]$, ensuring that confidence comparisons are well-behaved and preventing overflow in implementations.

Theorem E.4 (Boundedness)**.** For any well-formed chain C, the confidence $\gamma(C) \in [0, 1]$.

Proof of Theorem E.4. We proceed by case analysis and induction on the number of VALIDATE events in C.

Base case: $V = [e \in C \mid e.\textit{type} = \text{VALIDATE}] = []$. By definition, $\gamma(C) = 0.0 \in [0, 1]$.

Inductive step: Assume $\gamma(C) \in [0, 1]$ for a chain C with k distinct validators. Consider $C' = C \oplus [e_{\text{new}}]$ where $e_{\text{new}}.\textit{type} = \text{VALIDATE}$ and $e_{\text{new}}.\textit{actor}$ is a new actor (not in $\textit{actors}(V)$).

By definition:

$$\gamma(C') = \min\big(1.0,\ c'_{\text{base}} + 0.05 \times (N'_{\text{vals}} - 1)\big) \tag{24}$$

where $N'_{\text{vals}} = N_{\text{vals}} + 1 = k + 1$ and:

$$c'_{\text{base}} = \max\left(0.5,\ \frac{k \cdot \text{mean}_{a \in \textit{actors}(V)}\, \phi(a, V) + e_{\text{new}}.\textit{payload.confidence}}{k + 1}\right). \tag{25}$$

By the precondition for VALIDATE (Theorem E.6 and system invariants), $e_{\text{new}}.\textit{payload.confidence} \in [0, 1]$. The weighted mean is therefore a convex combination of values in $[0, 1]$, so it lies in $[0, 1]$. The $\max(0.5, \cdot)$ ensures $c'_{\text{base}} \geq 0.5$. The bonus term $0.05 \times k \geq 0$ is non-negative. Thus:

$$c'_{\text{base}} + 0.05 \times k \geq 0.5 > 0. \tag{26}$$

The outer $\min(1.0, \cdot)$ caps the value at 1.0. Therefore $\gamma(C') \in [0, 1]$.

If $e_{\text{new}}.\textit{actor} \in \textit{actors}(V)$ (duplicate validator), then $N'_{\text{vals}} = N_{\text{vals}}$ and c'_{base} uses argmax (retaining the higher confidence), so $\gamma(C') \geq \gamma(C) \geq 0$ and the $\min(1.0, \cdot)$ cap still applies.

Assumptions used: (A1) Individual confidence values $e.\textit{payload.confidence} \in [0, 1]$ for all VALIDATE events; (A2) The $\min(1.0, \cdot)$ construction is present in the definition of γ; (A3) The VALIDATE precondition requires confidence ≥ 0.5 (ensuring the lower bound). □

E.5 Theorem E.5: Monotonicity of Confidence

Confidence never decreases when a VALIDATE event is appended. This monotonicity property ensures that the concept governance process is cumulative: additional validation can only increase (or preserve) confidence.

Theorem E.5 (Monotonicity of Confidence)**.** Let C be a well-formed chain and let e_{new} be a VALIDATE event from an actor a with confidence $c \in [0, 1]$. Let $C' = C \oplus [e_{\text{new}}]$. Then:

$$\gamma(C') \geq \gamma(C). \tag{27}$$

Moreover, if $\gamma(C) < 1.0$ and a is a new validator, then $\gamma(C') > \gamma(C)$ (strict monotonicity).

Proof of Theorem E.5. Let $\gamma(C)$ be computed from V with $N_{\text{vals}} = |\mathit{actors}(V)|$ and base confidence c_{base}. We consider two cases.

Case 1: $a \notin \mathit{actors}(V)$ (new validator). Then $N'_{\text{vals}} = N_{\text{vals}} + 1$ and:

$$\gamma(C') = \min(1.0,\ c'_{\text{base}} + 0.05 \times N_{\text{vals}}) \tag{28}$$

where c'_{base} is the mean of $N_{\text{vals}} + 1$ per-actor maxima. The new base confidence is:

$$c'_{\text{base}} = \max\left(0.5,\ \frac{N_{\text{vals}} \cdot c^{\text{raw}}_{\text{base}} + c}{N_{\text{vals}} + 1}\right) \tag{29}$$

where $c^{\text{raw}}_{\text{base}} = \text{mean}_{a \in \mathit{actors}(V)}\, \phi(a, V)$. Even in the worst case where $c = 0$ (the minimum allowed by precondition is $c \geq 0.5$, but we prove the general case), the weighted mean satisfies:

$$\frac{N_{\text{vals}} \cdot c^{\text{raw}}_{\text{base}} + c}{N_{\text{vals}} + 1} \geq \frac{N_{\text{vals}} \cdot c^{\text{raw}}_{\text{base}}}{N_{\text{vals}} + 1}. \tag{30}$$

The bonus term increases from $0.05 \times (N_{\text{vals}} - 1)$ to $0.05 \times N_{\text{vals}}$, adding exactly 0.05. This increment dominates any possible decrease in the base confidence term because the base confidence is bounded below by 0.5 and the weighted mean of values in $[0, 1]$ cannot decrease by more than $1/(N_{\text{vals}} + 1) < 0.05$ for $N_{\text{vals}} \geq 1$.

Formally, let $\Delta = \gamma(C') - \gamma(C)$. If the $\min(1.0, \cdot)$ is not binding:

$$\Delta = (c'_{\text{base}} - c_{\text{base}}) + 0.05. \tag{31}$$

Since $c'_{\text{base}} \geq 0.5$ and $c_{\text{base}} \leq 1.0$, we have $c'_{\text{base}} - c_{\text{base}} \geq -0.5$, so $\Delta \geq -0.5 + 0.05 = -0.45$. But a tighter bound follows from the convexity of the mean: the decrease in base confidence is at most:

$$c_{\text{base}} - c'_{\text{base}} \leq \frac{c_{\text{base}} - c}{N_{\text{vals}} + 1} \leq \frac{1}{N_{\text{vals}} + 1} \leq 0.5 \quad (\text{for } N_{\text{vals}} \geq 1). \tag{32}$$

For $N_{\text{vals}} = 0$ (first validator), $\gamma(C) = 0$ and $\gamma(C') = \min(1.0, \max(0.5, c)) \geq 0.5 > 0 = \gamma(C)$. For $N_{\text{vals}} \geq 1$, the $+0.05$ bonus strictly dominates any possible decrease, so $\Delta > 0$ unless $\gamma(C) = 1.0$ (in which case $\gamma(C') = 1.0 = \gamma(C)$ by the min cap).

Case 2: $a \in \mathit{actors}(V)$ (duplicate validator). Then $N'_{\text{vals}} = N_{\text{vals}}$ and the bonus term is unchanged. The base confidence uses argmax:

$$\varphi'(a, V') = \max(\phi(a, V), c) \geq \phi(a, V). \tag{33}$$

Therefore $c'_{\text{base}} \geq c_{\text{base}}$, and since the bonus term is unchanged, $\gamma(C') \geq \gamma(C)$.

Assumptions used: (A1) Same actor's multiple VALIDATEs use argmax (highest confidence retained); (A2) The bonus term 0.05 per additional distinct validator is positive; (A3) Individual confidence values are non-negative. □

E.6 Theorem E.6: Status–Confidence Consistency

The final theorem establishes a critical consistency property between the status and confidence derivations: a concept with status *validated* must have positive confidence. This ensures that the status system and the confidence system are mutually coherent.

Theorem E.6 (Status–Confidence Consistency)**.** If $\delta(C) = \textit{validated}$, then $\gamma(C) > 0$.

Proof of Theorem E.6. Assume $\delta(C) = \textit{validated}$. By the definition of δ, this implies that the last lifecycle event in C is VALIDATE:

$$\text{last}(C_{\text{life}}).\textit{type} = \textsc{Validate}. \tag{34}$$

Therefore C_{life} is non-empty and contains at least one VALIDATE event. Consequently, the validation set $V = [e \in C \mid e.\textit{type} = \textsc{Validate}]$ is non-empty: $V \neq \emptyset$.

Since $V \neq \emptyset$, $\gamma(C)$ is computed by the second branch of its definition:

$$\gamma(C) = \min(1.0,\ c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1))\,. \tag{35}$$

By the VALIDATE precondition (system invariant), every VALIDATE event carries confidence $e.\textit{payload.confidence} \geq 0.5$. Therefore, for every actor $a \in \textit{actors}(V)$:

$$\phi(a, V) = \max\{e.\textit{payload.confidence} \mid e \in V \wedge e.a = a\} \geq 0.5. \tag{36}$$

The base confidence is:

$$c_{\text{base}} = \max\Big(0.5,\ \text{mean}_{a \in \textit{actors}(V)}\, \phi(a, V)\Big) \geq \max(0.5, 0.5) = 0.5. \tag{37}$$

Since $N_{\text{vals}} = |\textit{actors}(V)| \geq 1$ (at least one validator exists), the bonus term satisfies:

$$0.05 \times (N_{\text{vals}} - 1) \geq 0.05 \times 0 = 0. \tag{38}$$

Therefore:

$$\gamma(C) = \min(1.0,\ c_{\text{base}} + \text{bonus}) \geq \min(1.0,\ 0.5 + 0) = 0.5 > 0. \tag{39}$$

This completes the proof. In fact, we have established the stronger result that $\gamma(C) \geq 0.5$ whenever $\delta(C) = \textit{validated}$.

Assumptions used: (A1) The VALIDATE precondition requires confidence ≥ 0.5; (A2) $\delta(C) = \textit{validated}$ implies at least one VALIDATE event in C; (A3) The $\min(1.0, \cdot)$ cap does not bind below 0.5. □

E.7 Theorem 7: CRDT Convergence of EventChain Merge

The EventChain model supports an optional LWW-Set merge semantics for reconciling concurrent branches. This theorem establishes that the merge operation satisfies CRDT convergence properties.

Theorem E.7 (CRDT Convergence)**.** For any well-formed concurrent branches B_1, B_2 derived from the same genesis event (same concept_id and genesis hash), the LWW-Set merge operation $\text{Merge}(B_1, B_2) = B_1 \cup B_2$ (deduplicated by event_id) satisfies:

$$\text{Merge}(B_1, B_2) = \text{Merge}(B_2, B_1) \quad \text{(commutativity)} \tag{40}$$

$$\text{Merge}(\text{Merge}(B_1, B_2), B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3)) \quad \text{(associativity)} \tag{41}$$

$$\text{Merge}(B_1, B_1) = B_1 \quad \text{(idempotence)} \tag{42}$$

Furthermore, after merge, $\delta(\text{Merge}(B_1, B_2))$ is uniquely determined.

Proof of Theorem 7. We prove each CRDT property individually.

Commutativity. $\text{Merge}(B_1, B_2) = B_1 \cup B_2 = B_2 \cup B_1 = \text{Merge}(B_2, B_1)$ by commutativity of set union. Deduplication by event_id preserves this property because the deduplication function is symmetric: events with the same event_id appearing in both branches are included exactly once, regardless of which branch is the first argument.

Associativity. $\text{Merge}(\text{Merge}(B_1, B_2), B_3) = (B_1 \cup B_2) \cup B_3 = B_1 \cup (B_2 \cup B_3) = \text{Merge}(B_1, \text{Merge}(B_2, B_3))$ by associativity of set union. The deduplication step is idempotent: applying it to $(B_1 \cup B_2) \cup B_3$ yields the same result as applying it to $B_1 \cup (B_2 \cup B_3)$ because the set of distinct event_id values is the same in both cases.

Idempotence. $\text{Merge}(B_1, B_1) = B_1 \cup B_1 = B_1$ because each event in B_1 appears exactly once after deduplication by event_id, which is already satisfied within B_1 (Condition (5) of Φ ensures pairwise distinct event_id values).

Determinism of δ after merge. The merged set $B_1 \cup B_2$ may have a non-trivial lifecycle subsequence where events from both branches interleave. However, for equal-timestamp ties, event_id provides a total order (UUIDs are lexicographically comparable), making the last lifecycle event unique. By Theorem E.1, δ is therefore uniquely determined.

Assumptions used: (A1) Both B_1 and B_2 satisfy Φ (well-formedness); (A2) event_id values are pairwise distinct across all events in $B_1 \cup B_2$ except for shared ancestry events (which are deduplicated); (A3) UUID lexicographic ordering provides a total order for tie-breaking. □

E.8 Corollary: Event-Level CRDT (G-Set)

Corollary E.1 (Event-Level G-Set CRDT)**.** Each ADL Lite event e is a trivial CRDT: it is immutable (no update operation), so trivially convergent under any merge strategy that treats events as set elements. The EventChain C as an append-only log is a grow-only set (G-Set) CRDT Shapiro et al. [2011]: events can only be added, never removed, and the merge of two chains is the union of their event sets.

Proof of Corollary E.1. An event e is immutable by construction: the hash $e.h$ is computed over all event fields (including event_id and payload), and any modification would change the hash, breaking the cryptographic linkage. Therefore, events have no update operation—they are write-once. A G-Set with write-once elements is trivially a CRDT because there are no concurrent updates to resolve: all concurrent operations are disjoint additions to the set, and the merge is simply set union.

The EventChain C by design supports only the append operation ($C \oplus [e]$), not remove or update. Therefore, C satisfies the G-Set CRDT semantics: append is monotonic (the event set only grows), and Merge is union. The cryptographic hash linkage ensures that the order of events is preserved within each branch even after merge, though the interleaving of events from different branches is not deterministic without external arbitration (see Theorem 7 for tie-breaking via event_id ordering).

Assumptions used: (A1) Events are immutable (hash-locked); (A2) append is the only chain mutation operation; (A3) Set union with event_id deduplication is the merge operation. □

E.9 Summary of Proof Dependencies

The seven theorems and one corollary form a structured dependency graph:

- Theorem E.1 (Determinism) is foundational; no other theorem is used in its proof.
- Theorem E.2 (Confluence) depends on Theorem E.1.

- Theorem E.3 (Status Monotonicity) depends on Theorem E.1 and the ActionExecutor precondition system.
- Theorem E.4 (Boundedness) is independent of the others but shares assumptions with Theorem E.5.
- Theorem E.5 (Confidence Monotonicity) depends on Theorem E.4 for the cap argument and on the argmax semantics for duplicate validators.
- Theorem E.6 (Consistency) depends on Theorem E.4 (for the cap not interfering) and on the VALIDATE precondition system.
- Theorem E.7 (CRDT Convergence) depends on Theorem E.1 (for determinism of δ after merge) and on the well-formedness predicate Φ (for pairwise distinct event_id values).
- Corollary E.1 (G-Set CRDT) follows from the immutability of events and the append-only semantics of EventChains.

All seven proofs together establish that ADL Lite's derivation semantics is deterministic, confluent under fork, transition-monotonic, bounded, confidence-monotonic, internally consistent, and CRDT-convergent under LWW-Set merge. These properties provide the formal foundation for the system's operational correctness.

F RDF-star Interoperability Demonstration

G RDF-star Interoperability Demonstration

This appendix demonstrates the concrete interoperability path from ADL Lite EventChains to RDF-star, including export, query, and reversible mapping.

G.1 EventChain to PROV-O Export

Consider the EventChain for concept `disc-capital-trap` with two events: REGISTER (by `discoverer`) and VALIDATE (by `reviewer`, confidence 0.85). The PROV-O export is:

```
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix adl: <https://adl-lite.org/ns/> .

adl:evt-001 a prov:Activity ;
    prov:wasAssociatedWith adl:actor-discoverer ;
    adl:eventType "REGISTER" ;
    adl:previousEvent "genesis" ;
    adl:hash "sha256:abc123..." .

adl:evt-002 a prov:Activity ;
    prov:wasAssociatedWith adl:actor-reviewer ;
    prov:wasInformedBy adl:evt-001 ;
    adl:eventType "VALIDATE" ;
    adl:payloadConfidence 0.85 ;
    adl:hash "sha256:def456..." .
```

G.2 PROV-O to RDF-star Conversion

RDF-star enables statement-level annotation, which is natural for ADL Lite events (each event annotates a statement about the concept's lifecycle). The conversion:

```
# The concept's validation as an RDF-star triple
<< adl:disc-capital-trap adl:status "validated" >>
    adl:validator adl:actor-reviewer ;
    adl:confidence 0.85 ;
    adl:eventHash "sha256:def456..." ;
    adl:timestamp "2024-01-15T14:30:00Z" .

# A relation as RDF-star
<< adl:capital-trap adl:isomorphic-to adl:gradient-explosion >>
    adl:confidence 0.92 ;
    adl:mappingType "topological" ;
    adl:validator adl:actor-reviewer .
```

G.3 SPARQL-star Query

```
SELECT ?concept ?status ?validator ?confidence
WHERE {
    << ?concept adl:status ?status >>
        adl:validator ?validator ;
        adl:confidence ?confidence .
    FILTER (?confidence > 0.8)
}
```

This query retrieves all concepts with validated status and confidence > 0.8, including the validator identity and confidence value—information that requires reification in standard RDF but is first-class in RDF-star.

G.4 Reversible Mapping

The mapping ADL Lite $\rightarrow$ RDF-star is *lossy* in one direction: RDF-star discards the cryptographic hash chain (it captures the *semantics* of events but not their *integrity evidence*). However, a *reversible* mapping is possible by extending the RDF-star representation with provenance metadata:

```
<< adl:disc-capital-trap adl:status "validated" >>
    adl:provenance adl:evt-002 .

adl:evt-002 a adl:Event ;
    adl:hash "sha256:def456..." ;
    adl:previousEventHash "sha256:abc123..." .
```

With this extension, an ADL Lite system can: (1) export to RDF-star for SPARQL querying and Semantic Web interoperability, (2) import from RDF-star by reconstructing the EventChain from `adl:provenance` links, and (3) verify integrity by checking that the reconstructed chain's hashes match the stored `adl:hash` values.

References

IBM AML Simulated Dataset. https://www.kaggle.com/datasets/berkanozdemir/aml-simulated. Accessed: 2025-01-01.

Anonymous. Real-Time Collaboration in Linked Data Systems. In *Proceedings of the ISWC 2023 Posters, Demos and Industry Tracks*, volume 3632 of *CEUR Workshop Proceedings*, 2023. Applies CRDTs to RDF resource editing in Solid Pods.

Davide Francesco Barbieri, Daniele Braga, Stefano Ceri, Emanuele Della Valle, and Michael Grossniklaus. C-SPARQL: A Continuous Query Language for RDF Data Streams. In *ISWC*, 2009.

Miguel Castro and Barbara Liskov. Practical Byzantine Fault Tolerance. In *OSDI*, 1999.

ICOM-CIDOC. CIDOC Conceptual Reference Model. *ISO 21127:2014*, 2014.

Tobias Kuhn and Michel Dumontier. Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data. *Journal of Biomedical Semantics*, 6, 2015.

Tobias Kuhn, Christine Chichester, Michael Krauthammer, and Michel Dumontier. Nanopublications: A Growing Resource of Provenance-Centric Scientific Linked Data. In *IEEE eScience*, 2015.

LangChain, Inc. LangChain. https://www.langchain.com/, 2023.

Martin Kleppmann Martin et al. Automerge: A JSON CRDT for Real-Time Collaborative Editing. *ACM SIGOPS Operating Systems Review*, 2020.

Kevin Nicolai and Peter Dadam. Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types. *Proceedings of the ACM on Human-Computer Interaction*, 2021.

OpenAI. OpenAI Agents SDK. https://platform.openai.com/docs/, 2024.

Palantir Technologies. Palantir Foundry Data Engine. https://www.palantir.com/platforms/foundry/, 2024.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-Free Replicated Data Types. *Lecture Notes in Computer Science*, 6976, 2011.

Manu Sporny, Gregg Kellogg, and Markus Lanthaler. JSON-LD 1.1: A JSON-based Serialization for Linked Data. Technical report, W3C, 2020.

W3C. PROV-O: The PROV Ontology. https://www.w3.org/TR/prov-o/, 2013.

W3C. SHACL Shapes Constraint Language. https://www.w3.org/TR/shacl/, 2017.

W3C OWL Working Group. OWL 2 Web Ontology Language: Document Overview. In *W3C Recommendation*, 2012.

W3C RDF-star Working Group. RDF 1.2: RDF-star and SPARQL-star. *W3C Working Draft*, 2024.

W3C Verifiable Credentials Working Group. Verifiable Credential Data Integrity 1.0. *W3C Candidate Recommendation*, 2024.

Ludwig Wittgenstein. Tractatus Logico-Philosophicus. *Routledge & Kegan Paul*, 1922.

Greg Young. Event Sourcing: State from Events. *CodeBetter*, 2010.

Greg Young. Command and Query Responsibility Segregation (CQRS) Pattern. *Microsoft Patterns & Practices*, 2012.